

ORB-SLAM2源码解析

吴博

2019.12

- **预备知识**
- 视觉VO与重定位
- 局部优化
- 全局闭环

预备知识

- 1、3D-3D ICP与SE3/Sim3
- 2、3D-2D PnP相关算法
- 3、2D-3D Triangulation
- 4、2D-2D Homography/Fundamental
- 5、Bag-of-Words
- 6、Orb 特征提取与匹配

预备知识

1、3D-3D ICP与SE3/Sim3

2、3D-2D PnP相关算法

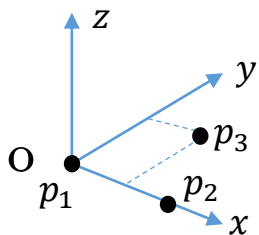
3、2D-3D Triangulation

4、2D-2D Homography/Fundamental

5、Bag-of-Words

6、Orb 特征提取与匹配

问题：已知n个匹配的3D点 $\{p_i\}$ 与 $\{q_i\}$, 求外参 sR, t , 使得 $p_i = sR \cdot q_i + t$



$$x = p_2 - p_1$$

$$\Rightarrow \bar{x} = x / \|x\|$$

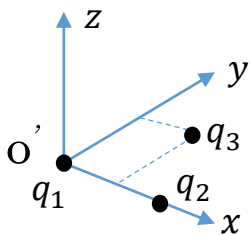
$$y = (p_3 - p_1) - [(p_3 - p_1) \cdot \bar{x}] \cdot \bar{x}$$

$$\Rightarrow \bar{y} = y / \|y\|$$

$$\bar{z} = \bar{x} \times \bar{y}$$

$$R_{w2o} = [\bar{x}, \bar{y}, \bar{z}]$$

$$R_{o2o'} = R_{w2o'} \cdot R_{w2o}^T$$



$$x' = q_2 - q_1$$

$$\Rightarrow \bar{x}' = x' / \|x'\|$$

$$y' = (q_3 - q_1) - [(q_3 - q_1) \cdot \bar{x}'] \cdot \bar{x}'$$

$$\Rightarrow \bar{y}' = y' / \|y'\|$$

$$\bar{z}' = \bar{x}' \times \bar{y}'$$

$$R_{w2o'} = [\bar{x}', \bar{y}', \bar{z}']$$

3D-3D ICP与SE3/Sim3

求Sim3尺度因子s

$$\begin{aligned}
 & \min_{R,t} \sum_{i=0}^{n-1} \| \overset{\text{平均值}}{\underset{\text{平均值}}{p_i}} - (s \cdot R \cdot q_i + t) \|^2 \\
 & \Rightarrow \sum_{i=0}^{n-1} \| (p_i' + p^-) - (s \cdot R \cdot (q_i' + q^-) + t) \|^2 \\
 & \Rightarrow \sum_{i=0}^{n-1} \| (p_i' - s \cdot R \cdot q_i') - (s \cdot R \cdot q^- - p^- + t) \|^2 \\
 & \Rightarrow \left(\sum_{i=0}^{n-1} \| p_i' - s \cdot R \cdot q_i' \|^2 \right) - 2t_0 \cdot \left(\sum_{i=0}^{n-1} (p_i' - s \cdot R \cdot q_i') \right) + n \cdot \| t_0 \|^2 \\
 & \quad \text{简写为 } t_0 \quad \text{等于0} \\
 & \Rightarrow \left(\sum_{i=0}^{n-1} \| p_i' - s \cdot R \cdot q_i' \|^2 \right) + n \cdot \| t_0 \|^2 \\
 & \quad \text{只有 } t_0 \text{ 和 } t \text{ 有关}
 \end{aligned}$$

如果是求SE3模型，则s=1

当 $t_0=0$ 时，得到最优 $t^* = p^- - sR \cdot q^-$ ，此时：

$$\begin{aligned}
 & \min \sum \| p_i' - s \cdot R \cdot q_i' \|^2 + n \cdot \| t_0 \|^2 \\
 & \Rightarrow \sum \| p_i' - s \cdot R \cdot q_i' \|^2 \\
 & \Rightarrow \sum \| p_i' \|^2 - 2s \cdot \sum p_i'^T \cdot R \cdot q_i' + s^2 \cdot \sum \| R \cdot q_i' \|^2 \\
 & \Rightarrow S_p - 2sD + s^2 S_q \\
 & s^* = D/S_q = \left(\sum p_i'^T \cdot R \cdot q_i' \right) / \sum \| R \cdot q_i' \|^2
 \end{aligned}$$

尺度计算的直观理解：模长之比：

$$\begin{array}{c}
 \xrightarrow{\text{red}} \xrightarrow{\text{blue}} \\
 R \cdot q_i' \quad p_i'
 \end{array}
 \quad s = p_i'^T \cdot R \cdot q_i' / \| q_i' \|^2$$

3D-3D ICP与SE3/Sim3

求Sim3尺度因子s

问题来了： $p_i = sR \cdot q_i + t$, 根据对称性 $q_i = \bar{s}\bar{R} \cdot p_i + \bar{t}$

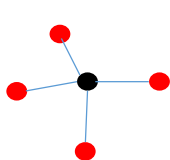
$$\text{可得: } \bar{s}^* = \left(\sum q_i'^T \cdot \bar{R} \cdot p_i' \right) / \sum \|\bar{R} \cdot p_i'\|^2$$

此时, $s^* \neq 1/\bar{s}^*$, 与理论不符合

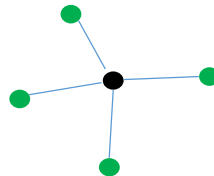
解决办法: 重新构造误差函数为 $e_i = \frac{1}{\sqrt{s}} p_i' - \sqrt{s} \cdot R \cdot p_i'$

此时, $s^* = \left[\sum \|p_i'\|^2 / \sum \|q_i'\|^2 \right]^{1/2}$, 且 $s^* = 1/\bar{s}^* \Rightarrow$ 计算过程不依赖 R, t

直观理解: 到中心点距离和之比



到中心点距离之和 $\sum \|p_i'\|^2$



到中心点距离之和 $\sum \|q_i'\|^2$

Similarity average

$$M = \sum_{i=0}^{n-1} p_i' \cdot q_i^T = \begin{bmatrix} S_{xx} & S_{xy} & S_{xz} \\ S_{yx} & S_{yy} & S_{yz} \\ S_{zx} & S_{zy} & S_{zz} \end{bmatrix}$$

SVD(M), 将 M 的特征值强设为1, 使得 $R \cdot R^T = I$

$$\overset{\text{SVD}}{M} = U \Sigma V^T \implies R = U \cdot V^T$$

$$N = \begin{bmatrix} S_{xx} + S_{yy} + S_{zz} & S_{yz} - S_{zy} & S_{zx} - S_{xz} & S_{xy} - S_{yx} \\ S_{yz} - S_{zy} & S_{xx} - S_{yy} - S_{zz} & S_{xy} + S_{yx} & S_{zx} + S_{xz} \\ S_{zx} - S_{xz} & S_{xy} + S_{yx} & -S_{xx} + S_{yy} - S_{zz} & S_{yz} + S_{zy} \\ S_{xy} - S_{yx} & S_{zx} + S_{xz} & S_{yz} + S_{zy} & -S_{xx} - S_{yy} + S_{zz} \end{bmatrix}$$

SVD(N), q^* 为最大奇异值对应的特征向量, 用罗德里格旋转公式转为 R

预备知识

1、3D-3D ICP与SE3/Sim3

2、3D-2D PnP相关算法

3、2D-3D Triangulation

4、2D-2D Homography/Fundamental

5、Bag-of-Words

6、Orb 特征提取与匹配

3D-2D PnP相关算法

问题：已知 n 个3D点 $\{ {}^w\mathbf{x}_i \}$ 以及与之匹配的2D点 $\{ u_i \}$ ，求外参 R, t

PnP算法大体分为直接法和优化法

常见的直接法包括：P3P、DLT、EPnP等

优化的算法包括：LHM、Only pos BA

PnP算法的指标主要包括：匹配点数、鲁棒性、速度、精度

P3P：3对匹配点，需要相机内参

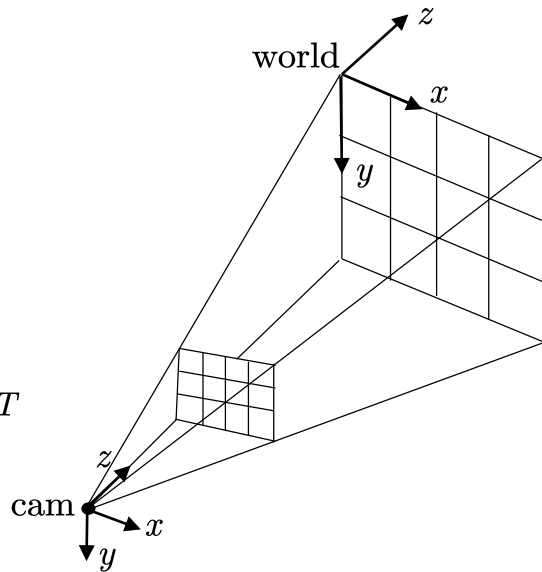
DLT：不需要内参，6点法求出单映矩阵，DLT分解出 K 、 R 、 T

EPnP：最小4个点，性价比高，精度较高，需要内参 K

LHM：复杂度较高，具有全局凸性，收敛性好，精度高

（在object坐标系建立误差模型）

only - pos BA：复杂度高，需要初始解，精度高（在image坐标系建立误差模型）



3D-2D PnP相关算法

EPnP算法

EPnP与P3P等直接法主要有两个步骤：

1. 利用刚体结构不变性，得到机体坐标系下3D点坐标
2. 根据世界坐标系和机体坐标系对应的3D点求解 R ， t

EPnP最大的特点：

用世界坐标系/机体坐标系下4个控制点表示所有世界坐标系/机体坐标系下3D点。再最小二乘求解后使用Gauss - Newton法优化控制点提高精度

3D-2D PnP相关算法

EPnP算法

步骤1：根据世界坐标系下3D点 $\{^w\mathbf{x}_i\}$ 用4个控制点构建质心坐标系

(1) $\{^w\mathbf{x}_i\}$ 求平均得到3D点中心点 $^w\mathbf{x}^-$ ，即 wc_0

(2) $\{^w\mathbf{x}_i\}$ 减去 $^w\mathbf{x}^-$ ，并构造 $C^T = \begin{bmatrix} ^w\mathbf{x}_0 - ^w\mathbf{x}^- \\ ^w\mathbf{x}_1 - ^w\mathbf{x}^- \\ \dots \\ ^w\mathbf{x}_{n-1} - ^w\mathbf{x}^- \end{bmatrix}$

(3) SVD(C)得到 wc_1 、 wc_2 、 wc_3

Lepetit, Vincent, Francesc Moreno-Noguer, and Pascal Fua. "EpnP: An accurate o (n) solution to the pnp problem." International journal of computer vision 81.2 (2009): 155.

3D-2D PnP相关算法

EPnP算法

步骤2：使用4个控制点构建的质心坐标系表示每一个 ${}^w\mathbf{x}_i$

$${}^w\mathbf{x}_i = \begin{bmatrix} {}^wc_0 & {}^wc_1 & {}^wc_2 & {}^wc_3 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}_i = \begin{bmatrix} {}^wc_0 & {}^wc_1 - {}^wc_0 & {}^wc_2 - {}^wc_0 & {}^wc_3 - {}^wc_0 \end{bmatrix} \begin{bmatrix} 1 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}_i$$

$$\Rightarrow ({}^w\mathbf{x}_i - {}^w\mathbf{x}^-) = \begin{bmatrix} {}^wc_1 - {}^wc_0 & {}^wc_2 - {}^wc_0 & {}^wc_3 - {}^wc_0 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}_i$$

$$= \begin{bmatrix} {}^wc_1' & {}^wc_2' & {}^wc_3' \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix}_i$$

$$\Rightarrow \begin{bmatrix} \alpha_1 & \alpha_2 & \alpha_3 \end{bmatrix}^T = \begin{bmatrix} {}^wc_1' & {}^wc_2' & {}^wc_3' \end{bmatrix}^{-1} ({}^w\mathbf{x}_i - {}^w\mathbf{x}^-)$$

$$\alpha_0 = 1 - \alpha_1 - \alpha_2 - \alpha_3$$

3D-2D PnP相关算法

EPnP算法

步骤3：求解机体坐标系下控制点

根据刚体变换结构不变性： ${}^w\alpha_{ij} = {}^c\alpha_{ij}$, $j = 0, 1, 2, 3$

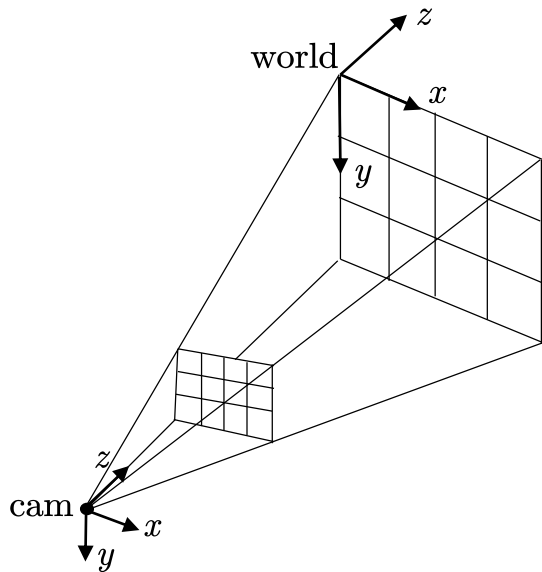
$$\begin{cases} u_i = ({}^c x_i / {}^c z_i) \cdot f_u + u \\ v_i = ({}^c y_i / {}^c z_i) \cdot f_v + v \end{cases} \Rightarrow \begin{cases} f_u \cdot {}^c x_i + (u - u_i) \cdot {}^c z_i = 0 \\ f_v \cdot {}^c y_i + (v - v_i) \cdot {}^c z_i = 0 \end{cases}$$

$$\Rightarrow \begin{cases} \sum_{j=0}^3 \alpha_{ij} \cdot f_u \cdot {}^c c_{xi} + \alpha_{ij} (u - u_i) \cdot {}^c c_{zi} = 0 \\ \sum_{j=0}^3 \alpha_{ij} \cdot f_v \cdot {}^c c_{yi} + \alpha_{ij} (v - v_i) \cdot {}^c c_{zi} = 0 \end{cases} \Rightarrow Mx = 0$$

$$M = \begin{bmatrix} \alpha_0 \cdot f_u & 0 & \alpha_0 \cdot (u_c - u_0), & \cdots, & \alpha_3 \cdot f_u & 0 & \alpha_3 \cdot (u_c - u_3) \\ 0 & \alpha_0 \cdot f_v & \alpha_0 \cdot (v_c - v_0), & \cdots, & 0 & \alpha_3 \cdot f_v & \alpha_3 \cdot (v_c - v_3) \end{bmatrix}_i$$

$$x = [{}^c c_0^T, {}^c c_1^T, {}^c c_2^T, {}^c c_3^T]$$

$$= [{}^c c_{x0} \ {}^c c_{y0} \ {}^c c_{z0}, {}^c c_{x1} \ {}^c c_{y1} \ {}^c c_{z1}, {}^c c_{x2} \ {}^c c_{y2} \ {}^c c_{z2}, {}^c c_{x3} \ {}^c c_{y3} \ {}^c c_{z3}]$$



SVD分解 M 得到特征向量 v_i ,

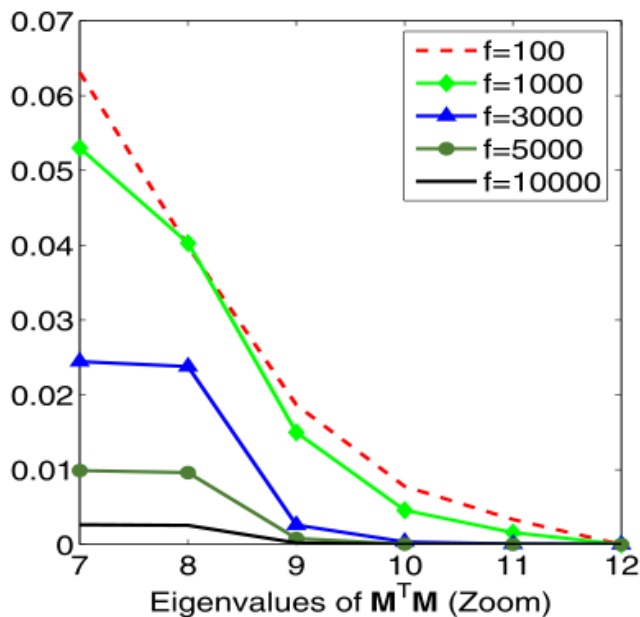
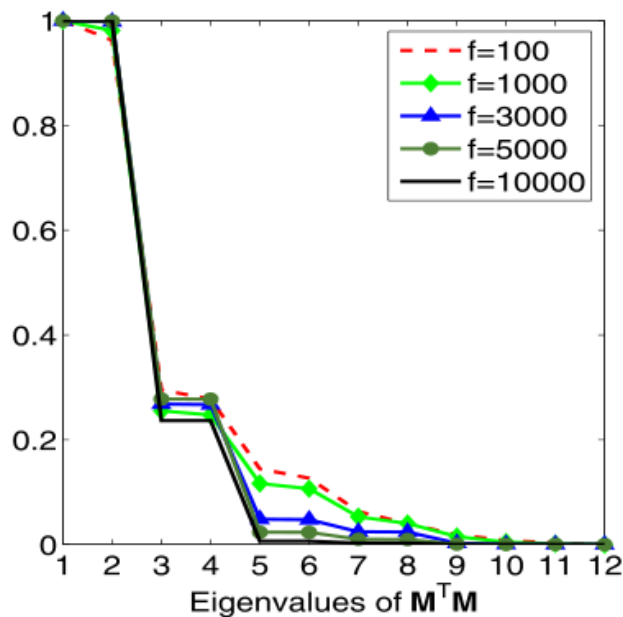
$$x = \sum_{i=1}^N \beta_i \cdot v_i$$

3D-2D PnP相关算法

EPnP算法

步骤4：确定 N 的数值和恢复 x 的尺度

Fig. 3 *Left:* Singular values of $\mathbf{M}^T \mathbf{M}$ for different focal lengths. Each curve averages 100 synthetic trials. *Right:* Zooming in on the smallest eigenvalues. For small focal lengths, the camera is perspective and only one eigenvalue is zero, which reflects the scale ambiguity. As the focal length increases and the camera becomes orthographic, all four smallest eigenvalues approach zero



3D-2D PnP相关算法

EPnP算法

步骤4：确定 N 的数值和(控制点间距离不变性)恢复 $\mathbf{x}$ 的尺度

$$\mathbf{x} = \sum_{i=1}^N \beta_i \mathbf{v}_i$$

$$\text{当 } N=1, \min_{k=(i,j), i,j \in (0,3), i < j} \sum_{k=1}^6 \|(\beta_1 \mathbf{v}_i - \beta_1 \mathbf{v}_j)^2 - ({}^w \mathbf{c}_i - {}^w \mathbf{c}_j)^2\|$$

$$\Rightarrow \beta_1 = \frac{\sum \|{}^w \mathbf{c}_i - {}^w \mathbf{c}_j\|}{\sum \|\mathbf{v}_i - \mathbf{v}_j\|}$$

$$\text{当 } N=2, \min_{k=(i,j), i,j \in (0,3), i < j} \sum_{k=1}^6 \|(\beta_1 \mathbf{v}_{1i} + \beta_2 \mathbf{v}_{2i} - (\beta_1 \mathbf{v}_{1j} + \beta_2 \mathbf{v}_{2j}))^2 - ({}^w \mathbf{c}_i - {}^w \mathbf{c}_j)^2\|$$

同理 $N=3, 4$

关于 β_1 、 β_2 、 β_3 、 β_4 的求解可采用密码学中的线性化方法

3D-2D PnP相关算法

EPnP算法

构造 β 的线性求解方程:

对于 $N=2,3,4$, 这里以一般的形式统一分析:

$$\min_{k=(i,j), i,j \in (1,4), i < j} \sum_{i=1}^6 \| {}^c D_{ij} - {}^w D_{ij} \|$$

$${}^w D_k = {}^w D_{ij} = \| {}^w c_i - {}^w c_j \|^2$$

$${}^c D_k = {}^c D_{ij} = \| \beta_1 v_{1i} + \beta_2 v_{2i} + \beta_3 v_{3i} + \beta_4 v_{4i} - (\beta_1 v_{1j} + \beta_2 v_{2j} + \beta_3 v_{3j} + \beta_4 v_{4j}) \|^2$$

$$= \| \beta_1 (v_{1i} - v_{1j}) + \beta_2 (v_{2i} - v_{2j}) + \beta_3 (v_{3i} - v_{3j}) + \beta_4 (v_{4i} - v_{4j}) \|^2$$

$$= \| \beta_1 \cdot dv_{1k} + \beta_2 \cdot dv_{2k} + \beta_3 \cdot dv_{3k} + \beta_4 \cdot dv_{4k} \|^2$$

$$= \beta_1^2 \cdot dv_{1k}^2 + 2\beta_1\beta_2 \cdot dv_{1k} \cdot dv_{2k} + \beta_2^2 \cdot dv_{2k}^2 + 2\beta_1\beta_3 \cdot dv_{1k} \cdot dv_{3k} + 2\beta_2\beta_3 \cdot dv_{2k} \cdot dv_{3k}$$

$$+ \beta_3^2 \cdot dv_{3k}^2 + 2\beta_1\beta_4 \cdot dv_{1k} \cdot dv_{4k} + 2\beta_2\beta_4 \cdot dv_{2k} \cdot dv_{4k} + 2\beta_3\beta_4 \cdot dv_{3k} \cdot dv_{4k} + \beta_4^2 \cdot dv_{4k}^2$$

即 $L_{6 \times 10} \cdot \beta_{10 \times 1} = \rho$, 其中:

$$L = [dv_1^2, 2dv_1 \cdot dv_2, dv_2^2, 2dv_1 \cdot dv_3, 2dv_2 \cdot dv_3, dv_3^2, 2dv_1 \cdot dv_4, 2dv_2 \cdot dv_4, 2dv_3 \cdot dv_4, dv_4^2]_k$$

$$\beta_l = [\beta_1^2, \beta_1\beta_2, \beta_2^2, \beta_1\beta_3, \beta_2\beta_3, \beta_3^2, \beta_1\beta_4, \beta_2\beta_4, \beta_3\beta_4, \beta_4^2]^T$$

3D-2D PnP相关算法

EPnP算法

得到 β 的近似解:

$$L = [dv_1^2, 2dv_1 \cdot dv_2, dv_2^2, 2dv_1 \cdot dv_3, 2dv_2 \cdot dv_3, dv_3^2, 2dv_1 \cdot dv_4, 2dv_2 \cdot dv_4, 2dv_3 \cdot dv_4, dv_4^2]_k$$

$$\beta_l = [\beta_{11}, \beta_{12}, \beta_{22}, \beta_{13}, \beta_{23}, \beta_{33}, \beta_{14}, \beta_{24}, \beta_{34}, \beta_{44}]^T$$

$$\text{近似解1: } \begin{cases} L = [dv_1^2, 2dv_1 \cdot dv_2, 2dv_1 \cdot dv_3, 2dv_1 \cdot dv_4] \\ \beta_l = [\beta_{11}, \beta_{12}, \beta_{13}, \beta_{14}]^T \end{cases}$$

$$\Rightarrow \beta_l \Rightarrow \beta = [\beta_1, \beta_2, \beta_3, \beta_4]^T = [|\sqrt{\beta_{11}}| \quad \beta_{12}/\beta_1 \quad \beta_{13}/\beta_1 \quad \beta_{14}/\beta_1]$$

$$\text{近似解2: } \begin{cases} L = [dv_1^2, 2dv_1 \cdot dv_2, 2dv_2 \cdot dv_2] \\ \beta_l = [\beta_{11}, \beta_{12}, \beta_{22}]^T \end{cases}$$

$$\Rightarrow \beta_l \Rightarrow \beta = [\beta_1, \beta_2, \beta_3, \beta_4]^T = [|\sqrt{\beta_{11}}| \quad |\sqrt{\beta_{22}}| \quad 0 \quad 0]$$

$$\text{近似解3: } \begin{cases} L = [dv_1^2, 2dv_1 \cdot dv_2, dv_2^2, 2dv_1 \cdot dv_3, 2dv_2 \cdot dv_3] \\ \beta_l = [\beta_{11}, \beta_{12}, \beta_{22}, \beta_{13}, \beta_{23}]^T \end{cases}$$

$$\Rightarrow \beta_l \Rightarrow \beta = [\beta_1, \beta_2, \beta_3, \beta_4]^T = [|\sqrt{\beta_{11}}| \quad |\sqrt{\beta_{22}}| \quad \beta_{13}/\beta_1 \quad 0]$$

3D-2D PnP相关算法

EPnP算法

Gauss – Newton迭代优化 β :

$$\begin{aligned} {}^c D_k = L \cdot \beta_l = & \beta_1^2 \cdot dv_{1k}^2 + 2\beta_1\beta_2 \cdot dv_{1k} \cdot dv_{2k} + \beta_2^2 \cdot dv_{2k}^2 \\ & + 2\beta_1\beta_3 \cdot dv_{1k} \cdot dv_{3k} + 2\beta_2\beta_3 \cdot dv_{2k} \cdot dv_{3k} \\ & + \beta_3^2 \cdot dv_{3k}^2 + 2\beta_1\beta_4 \cdot dv_{1k} \cdot dv_{4k} + 2\beta_2\beta_4 \cdot dv_{2k} \cdot dv_{4k} \\ & + 2\beta_3\beta_4 \cdot dv_{3k} \cdot dv_{4k} + \beta_4^2 \cdot dv_{4k}^2 \\ \Rightarrow J = \frac{\partial {}^c D_k}{\partial \beta} = & 2 \begin{bmatrix} dv_{1k}^2 + dv_{1k} \cdot dv_{2k} \cdot \beta_2 + dv_{1k} \cdot dv_{3k} \cdot \beta_3 + dv_{1k} \cdot dv_{4k} \cdot \beta_4 \\ dv_{1k} \cdot dv_{2k} \cdot \beta_1 + dv_{2k}^2 + dv_{2k} \cdot dv_{3k} \cdot \beta_3 + dv_{2k} \cdot dv_{4k} \cdot \beta_4 \\ dv_{1k} \cdot dv_{3k} \cdot \beta_1 + dv_{2k} \cdot dv_{3k} \cdot \beta_2 + dv_{3k}^2 + dv_{3k} \cdot dv_{4k} \cdot \beta_4 \\ dv_{1k} \cdot dv_{4k} \cdot \beta_1 + dv_{2k} \cdot dv_{4k} \cdot \beta_2 + dv_{3k} \cdot dv_{4k} \cdot \beta_3 + dv_{4k}^2 \end{bmatrix}^T \end{aligned}$$

最小二乘求解 β ， n 表示当前迭代次数:

$$J^{(n)} \cdot \beta^* = {}^w D_k - {}^c D_k^{(n-1)},$$

3D-2D PnP相关算法

P3P算法和DLT算法

DLT分解法求解PnP:

$$\begin{bmatrix} u_x \\ u_y \\ 1 \end{bmatrix} = \lambda K \cdot [R \ t] \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \lambda \begin{bmatrix} p_1 & p_2 & p_3 & p_4 \\ p_5 & p_6 & p_7 & p_8 \\ p_9 & p_{10} & p_{11} & p_{12} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \lambda \begin{bmatrix} \mathbf{p}_0^T \\ \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} \mathbf{x}$$

$$\Rightarrow u_x = \frac{\mathbf{p}_0^T \mathbf{x}}{\mathbf{p}_2^T \mathbf{x}}, u_y = \frac{\mathbf{p}_1^T \mathbf{x}}{\mathbf{p}_2^T \mathbf{x}} \Rightarrow \begin{cases} \mathbf{p}_0^T \mathbf{x} - \mathbf{p}_2^T \mathbf{x} \cdot u_x = 0 \\ \mathbf{p}_1^T \mathbf{x} - \mathbf{p}_2^T \mathbf{x} \cdot u_y = 0 \end{cases}$$

$$\Rightarrow \begin{bmatrix} \mathbf{x}^T & 0 & -\mathbf{x}^T u_x \\ 0 & \mathbf{x}^T & -\mathbf{x}^T u_y \end{bmatrix} \begin{bmatrix} \mathbf{p}_0 \\ \mathbf{p}_1 \\ \mathbf{p}_2 \end{bmatrix} = \mathbf{0} \Rightarrow \text{齐次最小二乘得到} \begin{bmatrix} \mathbf{p}_0 \\ \mathbf{p}_1 \\ \mathbf{p}_2 \end{bmatrix}$$

$$\text{投影矩阵 } P = K \cdot [R \ t] = \begin{bmatrix} p_1 & p_2 & p_3 & p_4 \\ p_5 & p_6 & p_7 & p_8 \\ p_9 & p_{10} & p_{11} & p_{12} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_0^T \\ \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix}$$

$$P = K \cdot [R \ t] = K \cdot [R \ -Rc] = [M \ -Mc]$$

$$M = QR \Rightarrow K, R \Rightarrow t$$

P3P方法见：《视觉slam十四讲从理论到实践》

预备知识

1、3D-3D ICP与SE3/Sim3

2、3D-2D PnP相关算法

3、2D-3D Triangulation

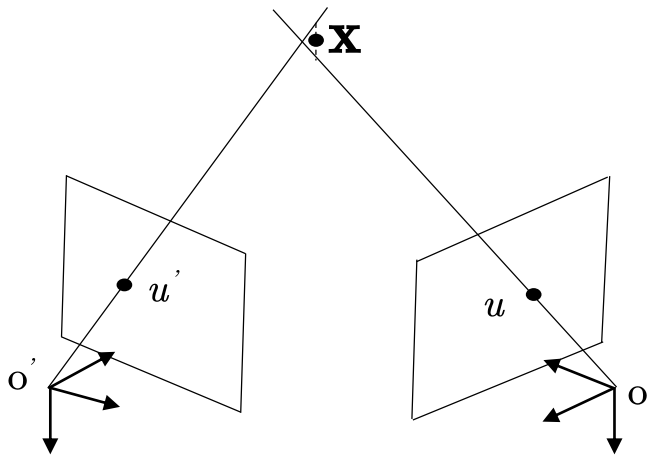
4、2D-2D Homography/Fundamental

5、Bag-of-Words

6、Orb 特征提取与匹配

2D-3D Triangulation

问题：已知多个视角匹配的2D点 $\{u, u'\}$ ，多个视角相机的投影矩阵 $\{P, P'\}$ ，求3D点



$$\begin{bmatrix} u_x \\ u_y \\ 1 \end{bmatrix} = \lambda K \cdot [R \ t] = \lambda \begin{bmatrix} p_1 & p_2 & p_3 & p_4 \\ p_5 & p_6 & p_7 & p_8 \\ p_9 & p_{10} & p_{11} & p_{12} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

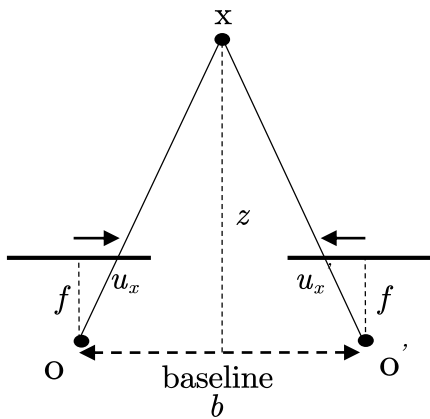
$$= \lambda \begin{bmatrix} \mathbf{p}_0^T \\ \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} \mathbf{x}$$

$$u = \lambda P \mathbf{x}, \quad \text{DLT} \Rightarrow \begin{bmatrix} u_y \cdot \mathbf{p}_2^T - \mathbf{p}_1^T \\ \mathbf{p}_0^T - u_x \mathbf{p}_2^T \\ u_x \cdot \mathbf{p}_1^T - u_y \cdot \mathbf{p}_0^T \end{bmatrix} \mathbf{x} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{一对匹配点} \begin{cases} u = \lambda P \mathbf{x} \\ u' = \lambda' P' \mathbf{x} \end{cases} \Rightarrow \begin{bmatrix} u_y \cdot \mathbf{p}_2^T - \mathbf{p}_1^T \\ \mathbf{p}_0^T - u_x \mathbf{p}_2^T \\ u_y' \cdot \mathbf{p}_2'^T - \mathbf{p}_1'^T \\ \mathbf{p}_0'^T - u_x' \mathbf{p}_2'^T \end{bmatrix} \mathbf{x} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (\text{SVD分解求解})$$

2D-3D Triangulation

当两个相机之间无相对旋转，且相机中心连线与x轴平行，则称为双目立体视觉



$$\mathbf{x} = [x \ y \ z]^T$$

$$\begin{cases} \frac{x}{z} = \frac{u_x}{f} \\ \frac{b-x}{z} = \frac{-u_x'}{f} \end{cases}$$

$$\Rightarrow d = u_x - u_x' = \frac{f \cdot b}{z}$$

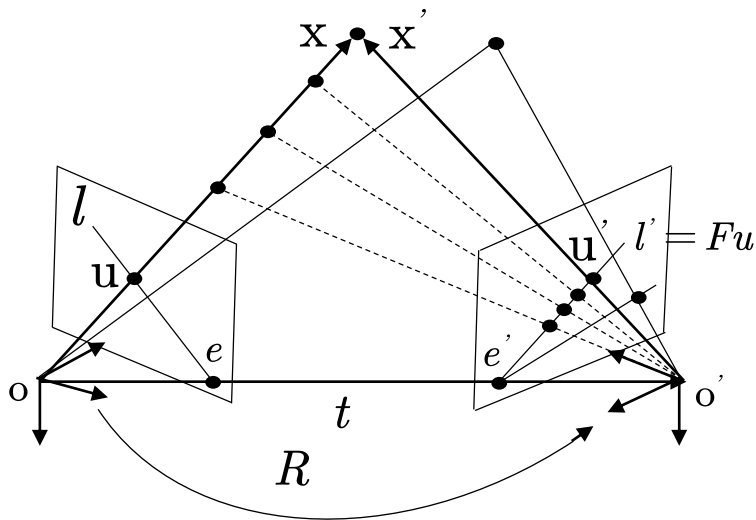
$$\Rightarrow \begin{cases} z = \frac{f \cdot b}{d} \\ x = \frac{u_x}{f} \cdot z \\ y = \frac{u_y}{f} \cdot z \end{cases}$$

注：对于双目结构，对极几何的极点在左右无穷远处

预备知识

- 1、3D-3D ICP与SE3/Sim3
- 2、3D-2D PnP相关算法
- 3、2D-3D Triangulation
- 4、2D-2D Homography/Fundamental**
- 5、Bag-of-Words
- 6、Orb 特征提取与匹配

2D-2D 对极几何



$$e' = P' \cdot o, P \cdot o = 0$$

$$P' = K' [R | t'], P = K [R | t]$$

$$l' = e' \times u = (P' \cdot o) \times (P' P^{-1} u) = F u$$

$$\Rightarrow F = [e']_{\times} P' P^{-1}$$

Fundamental模型

o和o'相机坐标系之间存在刚体变换: $x' = R \cdot (x - t)$ (1)

根据共平面: $(x - t)^T (t \times x) = 0$ (2)

$$(1) (2) \Rightarrow (x'^T \cdot R) (t \times x) = 0$$

$$\Rightarrow (x'^T \cdot R) ([t]_{\times} \cdot x) = 0$$

$$\Rightarrow x'^T (R \cdot [t]_{\times}) x = 0$$

$$\Rightarrow \mathbf{x}'^T E \mathbf{x} = 0 \Rightarrow \bar{\mathbf{x}}'^T E \bar{\mathbf{x}} = 0$$

x' 和 x 为同一个3D点在o和o'坐标系下坐标,

$\bar{x}'$ 和 $\bar{x}$ 为同一个3D点在o和o'归一化相机平面坐标系下坐标,

$$u = K \cdot x \Rightarrow \lambda_1 K_1^{-1} u = x, \lambda_2 K_2^{-1} u' = x'$$

$$x'^T E x = 0 \Rightarrow u'^T K_2^{-T} E K_1^{-1} u = 0$$

$$\Rightarrow u'^T F u = 0$$

极线 $l = F u$, 不同深度的3D点满足极线约束

已知极点的情况下, 只需要一个3D点即可确定 l 到 l' 的单应
不同的极线相交于极点 e , e 是相机 o' 投影到 o 坐标系的坐标点

2D-2D 对极几何

归一化变换

在DLT求解 F 和 H 矩阵之前对特征点归一化非常重要。

尤其对于特征点中心偏离中心点以及分布不均匀的情况。

归一化的作用：1. 特征点中心在图像中心处

2. 特征点各个方向平均值相等，即到图像中心点的距离为 $\sqrt{2}$

特征点 $\mathbf{u}_i = (u_x, u_y)_i$ 归一化步骤：

step1：求平均值 $u_x^- = (\sum u_x)/N$, $u_y^- = (\sum u_y)/N$, N 为特征点个数

step2：尺度缩放 $du_x^- = (\sum(u_x - u_x^-))/N$, $du_y^- = (\sum(u_y - u_y^-))/N$

归一化后的坐标 $\mathbf{u}_{n_i} = (u_{x_n}, u_{y_n})_i$ 为： $u_{x_n} = (u_x - u_x^-)/du_x^-$, $u_{y_n} = (u_y - u_y^-)/du_y^-$

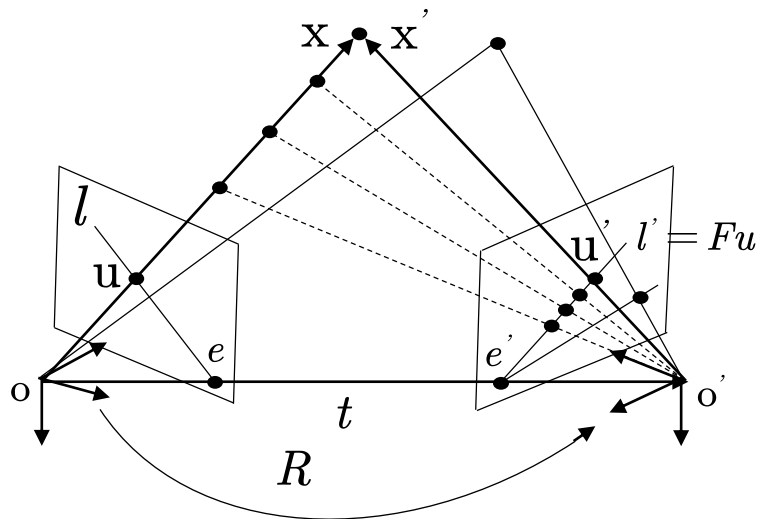
由上，特征点归一化的矩阵可以总结为：

$$T = \begin{bmatrix} s_x & 0 & -s_x u_x^- \\ 0 & s_y & -s_y u_y^- \\ 0 & 0 & 1 \end{bmatrix}, \text{ 其中 } s_x = 1/du_x^-, \quad s_y = 1/du_y^-$$

使用DLT求解 $\mathbf{u}_n'^T \bar{F} \mathbf{u}_n = 0$ ，则 $F = T_2^T \bar{F} T_1$

2D-2D 对极几何

Fundamental模型求解



“归一化”8点法：不共面8对图像匹配点，见备注

$$u'^T F u = 0, F = \begin{bmatrix} f_1 & f_2 & f_3 \\ f_4 & f_5 & f_6 \\ f_7 & f_8 & f_9 \end{bmatrix}$$

经过线性变换 $\Rightarrow A \cdot f = 0$

其中： $A = [u'_x u_x \quad u'_x u_y \quad u'_x \quad u'_y u_x \quad u'_y u_y \quad u'_y \quad u_x \quad u_y \quad 1]$

$$f = [f_1 \quad f_2 \quad f_3 \quad f_4 \quad f_5 \quad f_6 \quad f_7 \quad f_8 \quad f_9]^T$$

SVD(A)最小奇异值对应的特征向量

当3D点都共平面时：

$\bar{x}' = H \bar{x}$ ，任取一个向量 n

$$\Rightarrow (n \times \bar{x}') \perp (H \bar{x}) = 0$$

$\Rightarrow \bar{x}' [n]_{\times} H \bar{x} = 0 \Rightarrow F = [n]_{\times} H$ ，存在无穷多个
引用自白巧克力亦唯心“Homography知多少”

备注：7点法求 F 矩阵见《计算机视觉中的多视图几何》p192 10.1.2 小节

2D-2D 对极几何

Fundamental奇异值约束

DLT求解 F 后需要根据奇异值约束(F 矩阵自由度为7, 秩为2)

优化Frobenius范数调整 F :

$$\min ||F^* - F||$$

s.t. SVD(F) 奇异值满足 $[\sigma_1, \sigma_2, 0]$

$$\Rightarrow F^* = U \text{diag}(\sigma_1, \sigma_2, 0) V^T$$

$E = K_2^T F K_1 = U \Sigma V^T$, 需要进行奇异值约束:

$$E = U \text{diag}\left(\frac{\sigma_1 + \sigma_2}{2}, \frac{\sigma_1 + \sigma_2}{2}, 0\right) V^T, \text{ 自由度为5, 见备注}$$

备注: F 矩阵的奇异值约束, 见《计算机视觉中的多视图几何》p192 10.1.2 节

为什么 E 矩阵SVD分解后有两个相等的奇异值, 见《计算机视觉中的多视图几何》p174 结论8.17

2D-2D 对极几何

Fundamental模型分解

$E = U\Sigma V^T$, Σ 经过约束处理

$$\text{取 } W = R_z(\pi/2) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

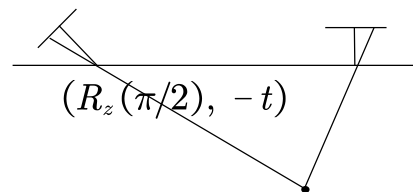
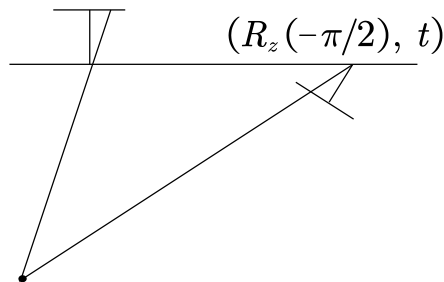
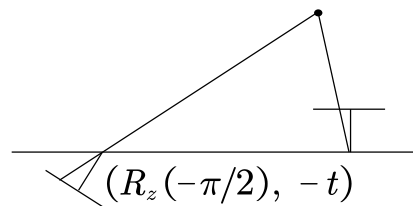
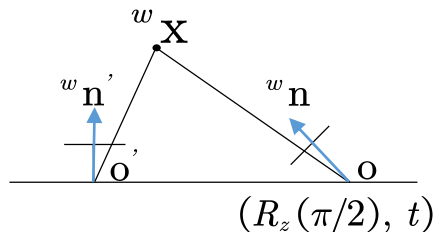
$$\begin{cases} R_1 = UWV^T \\ R_2 = UW^TV^T \end{cases} \quad \begin{cases} T_1 = U_3 \\ T_2 = -U_3 \end{cases}$$

4个解中根据如下条件选择最优解:

(1) 恢复的3D点的相机前方

$$\begin{cases} {}^w\mathbf{n}^T \cdot ({}^w\mathbf{x} - \mathbf{o}) > 0 \\ {}^{w'}\mathbf{n}'^T \cdot ({}^w\mathbf{x} - \mathbf{o}') > 0 \end{cases}$$

(2) 视差角最大的模型 (视差角: $\angle \mathbf{o}' \mathbf{x} \mathbf{o}$)



备注: E 分解 R 和 t 的推导可见《计算机视觉中的多视图几何》p174 结论8.18

2D-2D 单应变换

Homography模型

平面法向量为 $n = (a, b, c)^T$

若一个3D点 $x = (x, y, z)^T$ 在平面上, 则:

$$a \cdot x + b \cdot y + c \cdot z = d, \text{ 即 } \frac{1}{d} \cdot n^T \cdot x = 1$$

其中 d 表示3D点到平面的垂直距离

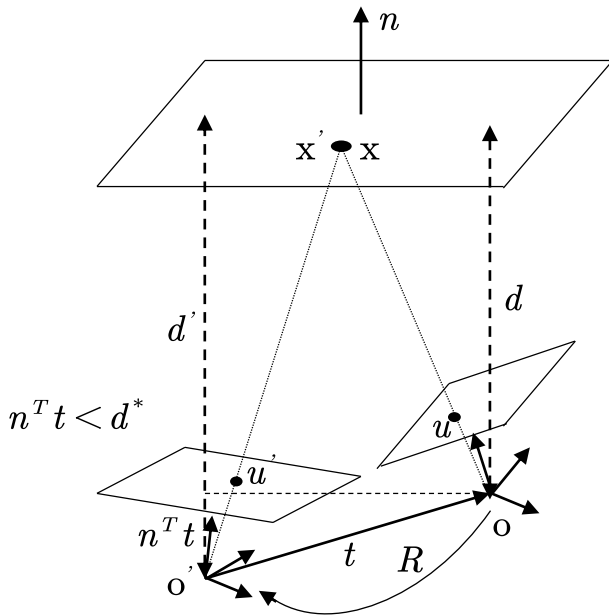
$$x' = R \cdot x + t$$

用 $\bar{x}'$ 和 $\bar{x}$ 表示归一化相机坐标系:

$$\begin{aligned} \lambda_2 \cdot \bar{x}' &= R_{o_2 o_1} \cdot (\lambda_1 \cdot \bar{x}) + t_{o_2 o_1} \\ &= R_{o_2 o_1} \cdot (\lambda_1 \cdot \bar{x}) + t_{o_2 o_1} \cdot 1 \\ &= R_{o_2 o_1} \cdot (\lambda_1 \cdot \bar{x}) + t_{o_2 o_1} \cdot \frac{1}{d} \cdot n^T \cdot (\lambda_1 \cdot \bar{x}) \end{aligned}$$

$$\Rightarrow \bar{x}' = \lambda \cdot \left(R_{o_2 o_1} + \frac{1}{d} \cdot t_{o_2 o_1} \cdot n^T \right) \bar{x} = \hat{H} \cdot \bar{x}$$

对应到图像坐标系: $u' = G \cdot u, G = K_o \hat{H} K_{o_1}^{-1}$



由公式可知:

相机纯旋转后图像满足单映变换
在工程上 d 远大于 t 时可认为纯旋转
此时称为无穷单应 H_∞

2D-2D 单应变换

DLT方法求解单映矩阵:

$$\bar{\mathbf{x}}' = \hat{H} \cdot \bar{\mathbf{x}}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\Rightarrow \begin{cases} x' = \frac{kh_1x + kh_2y + kh_3}{kh_7 + kh_8y + kh_9} \\ y' = \frac{kh_4x + kh_5y + kh_6}{kh_7 + kh_8y + kh_9} \end{cases}$$

故: $\hat{H}$ 矩阵自由度为8

Homography模型求解

1. 通过强制约束 $h_9=1$ 来约束自由度为8:

通过左侧推导 $A \cdot h = b$

$$\text{其中: } A = \begin{bmatrix} 0 & 0 & 0 & -x & -y & -1 & xy' & yy' \\ -x & -y & -1 & 0 & 0 & 0 & xx' & yx' \end{bmatrix}$$

$$b = [-y' \quad -x']^T$$

$$h = [h_1 \quad h_2 \quad h_3 \quad h_4 \quad h_5 \quad h_6 \quad h_7 \quad h_8]^T$$

四点法: $A_{8 \times 8} \cdot h_{8 \times 1} = b_{8 \times 1}$

但是, 根据 H 模型的定义, 存在 $h_9=0$ 的可能性, 则存在数值问题

2. 约束 $h_1^2 + h_2^2 + h_3^2 + h_4^2 + h_5^2 + h_6^2 + h_7^2 + h_8^2 + h_9^2 = 1$

通过左侧推导或DLT方法: $\bar{\mathbf{x}}' \times \hat{H} \cdot \bar{\mathbf{x}} = 0 \Rightarrow A \cdot h = 0$

$$\text{其中: } A = \begin{bmatrix} 0 & 0 & 0 & -x & -y & -1 & xy' & yy' & y' \\ -x & -y & -1 & 0 & 0 & 0 & xx' & yx' & x' \end{bmatrix}$$

$$h = [h_1 \quad h_2 \quad h_3 \quad h_4 \quad h_5 \quad h_6 \quad h_7 \quad h_8 \quad h_9]^T$$

四点法: $A_{8 \times 9} \cdot h_{9 \times 1} = 0 \Rightarrow \text{SVD}(A)$ 得到最优 $\hat{H}$

2D-2D 单应变换

Homography模型分解

常见有三种方法：

基于SVD分解（2种）： $\begin{cases} \text{Faugeras SVD (Orbslam2)} \\ \text{Zhang - SVD (OpenCV), 见ppt最后补充材料} \end{cases}$

直接解析求解：Ezio Malis方法 (OpenCV)，见ppt最后补充材料

Faugeras SVD方法：将 $H = UAV^T$ 进行SVD分解， A 也可以认为是单映矩阵
并构造3个方程，再利用旋转矩阵的正交性，可先求出平面法向量，
再求解出旋转矩阵和平移量

Zhang SVD方法：对 $H^T H = V\Lambda V^T$ 进行SVD分解，根据 $((R^T t) \times n)$ 对应特征
值等于1的特征向量，并利用 $H^T H$ 特征向量的正交性进行推导分解过程

Ezio Malis方法：利用 $H^T H$ 的对称性，并转换为先求解 $x = R^T t / \|t\|$, $y = \|t\|n$
两个待求向量，进而分解出 n, R, t

2D-2D 单应变换

Homography模型分解

Faugeras SVD

$$\hat{H} = R_{o_2o_1} + \frac{1}{d} \cdot t_{o_2o_1} \cdot n^T \cong d \cdot R + t \cdot n^T = U \Lambda V^T$$

$$\Rightarrow \Lambda = U^T \hat{H} V = d U^T R V + (U^T t) (V^T n)^T$$

$$\text{设 } s = \det(U) \cdot \det(V), s^2 = 1$$

$$\begin{aligned} \Rightarrow \Lambda &= s^2 d U^T R V + (U^T t) (V^T n)^T \\ &= (sd) (s U^T R V) + (U^T t) (V^T n)^T \end{aligned}$$

$$\text{令: } R' = s U^T R V, t' = U^T t, n' = V^T n, d' = sd$$

$$\text{则: } \Lambda = d' R' + t' n'^T, \text{ 故 } \Lambda \text{ 也可以认为是单应矩阵}$$

备注: 引入 s 是为了说明 R' 与 R , d' 与 d 存在符号取反的可能

$$\Lambda = \begin{bmatrix} d_1 & 0 & 0 \\ 0 & d_2 & 0 \\ 0 & 0 & d_3 \end{bmatrix} = d' R' + t' n'^T \quad (1)$$

$$\text{取: } e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, e_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, e_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\text{则: } n' = \begin{bmatrix} x_1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ x_2 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ x_3 \end{bmatrix} = x_1 e_1 + x_2 e_2 + x_3 e_3$$

$$\begin{aligned} (1) \Rightarrow [d_1 e_1, d_2 e_2, d_3 e_3] &= [d' R' e_1, d' R' e_2, d' R' e_3] \\ &\quad + [t' x_1, t' x_2, t' x_3] \end{aligned}$$

$$\Rightarrow \begin{cases} d_1 e_1 = d' R' e_1 + t' x_1 & (2) \\ d_2 e_2 = d' R' e_2 + t' x_2 & (3) \\ d_3 e_3 = d' R' e_3 + t' x_3 & (4) \end{cases}$$

2D-2D 单应变换

Homography模型分解

Faugeras SVD

(2) (3) (4)中每两个式子消去 t' 可得:

$$\begin{cases} d'R'(x_2e_1 - x_1e_2) = d_1x_2e_1 - d_2x_1e_2 \\ d'R'(x_3e_2 - x_2e_3) = d_2x_3e_2 - d_3x_2e_3 \\ d'R'(x_1e_3 - x_3e_1) = d_3x_1e_3 - d_1x_3e_1 \end{cases} \quad (5)$$

由于: $\|R'x\| = \|x\|$

对(5)式子中三个式子的左右两边同时取二范数可得:

$$\begin{cases} (d'^2 - d_2^2)x_1^2 + (d'^2 - d_1^2)x_2^2 = 0 \\ (d'^2 - d_3^2)x_2^2 + (d'^2 - d_2^2)x_3^2 = 0 \\ (d'^2 - d_1^2)x_3^2 + (d'^2 - d_3^2)x_1^2 = 0 \end{cases} \quad (6)$$

令: $d'^2 - d_1^2 = a, d'^2 - d_2^2 = b, d'^2 - d_3^2 = c$

$$(6) \Rightarrow \begin{bmatrix} b & a & 0 \\ 0 & c & b \\ c & 0 & a \end{bmatrix} \begin{bmatrix} x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix} = 0 \Rightarrow \det \begin{pmatrix} b & a & 0 \\ 0 & c & b \\ c & 0 & a \end{pmatrix} = 0 \Rightarrow abc = 0$$

$$\Rightarrow (d'^2 - d_1^2)(d'^2 - d_2^2)(d'^2 - d_3^2) = 0 \quad (7)$$

$$\Rightarrow d' = \pm d_2$$

证明: $d' = \pm d_2$

由于 $d_1 \geq d_2 \geq d_3$, (7)式可以分成三种情况:

$$\begin{cases} \textcircled{1} d_1 \neq d_2 \neq d_3 \\ \textcircled{2} d_1 = d_2 \neq d_3 \text{ or } d_1 \neq d_2 = d_3 \\ \textcircled{3} d_1 = d_2 = d_3 \end{cases}$$

对第1中情况进行反证:

如果 $d' \neq \pm d_2$, 则 $d' = \pm d_1$ 或 $d' = \pm d_3$

根据(6)式:

$$\begin{cases} x_1 = 0 \\ (d_1^2 - d_3^2)x_2^2 + (d_1^2 - d_2^2)x_3^2 = 0 \\ d_1 > d_2 > d_3 \text{ (SVD特征值降序排列)} \end{cases}$$

$\Rightarrow x_1 = x_2 = x_3 = 0$ 与 $x_1^2 + x_2^2 + x_3^2 = 1$ 矛盾

2D-2D 单应变换

Homography模型分解

Faugeras SVD

假设 $d' = d_2 > 0$ ，则根据(6)式以及 $x_1^2 + x_2^2 + x_3^2 = 1$ ：

$$n' = \begin{cases} x_1 = \varepsilon_1 \cdot \sqrt{\frac{d_1^2 - d_2^2}{d_1^2 - d_3^2}} \\ x_2 = 0 \\ x_3 = \varepsilon_2 \cdot \sqrt{\frac{d_2^2 - d_3^2}{d_1^2 - d_3^2}} \\ \varepsilon_1, \varepsilon_2 = \pm 1 \end{cases}$$

$$(8) \Rightarrow \begin{cases} \sin \theta = \frac{(d_1 - d_3)}{d_2} x_1 x_3 = \varepsilon_3 \cdot \frac{\sqrt{(d_1^2 - d_2^2)(d_2^2 - d_3^2)}}{(d_1 + d_3)d_2} \\ \cos \theta = \frac{d_3 x_1^2 + d_1 x_3^2}{d_2} = \frac{d_1 d_3 + d_2^2}{(d_1 + d_3)d_2} \end{cases}$$

$$\text{根据(2)(3)(4)} \Rightarrow t' = (d_1 - d_3) \begin{pmatrix} x_1 \\ 0 \\ x_3 \end{pmatrix}$$

带入(3)式 $\Rightarrow R'e_2 = e_2$

$$\Rightarrow R' = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}$$

将 n' 和 R' 带入(5)第3个式子：

$$d' \cdot \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} -x_3 \\ 0 \\ x_1 \end{bmatrix} = \begin{bmatrix} -d_1 x_3 \\ 0 \\ d_3 x_1 \end{bmatrix} \quad (8)$$

根据相机都在平面同一面 $\Rightarrow n^T t < d^*$ ，
因而只有这四组符合实际情况：

$$\begin{cases} \varepsilon_1 = 1, \varepsilon_2 = -1, \varepsilon_3 = 1 \\ \varepsilon_1 = 1, \varepsilon_2 = 1, \varepsilon_3 = -1 \\ \varepsilon_1 = -1, \varepsilon_2 = -1, \varepsilon_3 = -1 \\ \varepsilon_1 = -1, \varepsilon_2 = 1, \varepsilon_3 = 1 \end{cases}$$

2D-2D 单应变换

Homography模型分解

Faugeras SVD

同理，假设 $d' = -d_2 < 0$ ，则根据(6)式：

$$n' = \begin{cases} x_1 = \varepsilon_1 \cdot \sqrt{\frac{d_1^2 - d_2^2}{d_1^2 - d_3^2}} \\ x_2 = 0 \\ x_3 = \varepsilon_2 \cdot \sqrt{\frac{d_2^2 - d_3^2}{d_1^2 - d_3^2}} \\ \varepsilon_1, \varepsilon_2 = \pm 1 \end{cases}$$

$R'e_2 = -e_2$ ，则：

$$R' = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & -1 & 0 \\ \sin\theta & 0 & -\cos\theta \end{bmatrix}$$

$$t' = (d_1 + d_3) \begin{pmatrix} x_1 \\ 0 \\ x_3 \end{pmatrix}$$

$$\begin{cases} \sin\theta = \frac{(d_1 + d_3)}{d_2} x_1 x_3 = \varepsilon_3 \cdot \frac{\sqrt{(d_1^2 - d_2^2)(d_2^2 - d_3^2)}}{(d_1 - d_3)d_2} \\ \cos\theta = \frac{d_3 x_1^2 - d_1 x_3^2}{d_2} = \frac{d_1 d_3 - d_2^2}{(d_1 - d_3)d_2} \end{cases}$$

根据相机都在平面同一面 $\Rightarrow n^T t < d^*$ ，
因而只有这四组符合实际情况：

$$\begin{cases} \varepsilon_1 = 1, \varepsilon_2 = 1, \varepsilon_3 = 1 \\ \varepsilon_1 = 1, \varepsilon_2 = -1, \varepsilon_3 = -1 \\ \varepsilon_1 = -1, \varepsilon_2 = 1, \varepsilon_3 = -1 \\ \varepsilon_1 = -1, \varepsilon_2 = -1, \varepsilon_3 = 1 \end{cases}$$

2D-2D 模型约束与外点剔除

Ransac:

p : 内点的概率(比例)

N : 总样本数

K : 一次Ransac所需要的样本数

$\Rightarrow p^K$: 选取 K 个内点的概率

$\Rightarrow 1 - p^K$: K 个点中有一个外点的概率

$\Rightarrow (1 - p^K)^M$: M 次采样都失败的概率

$\Rightarrow p_s = 1 - (1 - p^K)^M$: M 次采样至少有一次成功的概率

$\Rightarrow M = \frac{\log(1 - p_s)}{\log(1 - p^K)}$: 要达到 p_s 的成功率所需要的采样次数

外点剔除步骤:

选择集合约束模型(F或者H)

随机采样匹配点对模型进行求解

① 8点法求F模型矩阵

② 4点法求H模型矩阵

③ 2点法(已知旋转)求t

根据求解的模型计算误差(见备注), 并统计内点个数

① F模型: $e = \frac{(u'^T F u)^2}{(Fu)_x^2 + (Fu)_y^2 + (u'^T F)_x^2 + (u'^T F)_y^2}$

② H模型几何距离: $e = (u' - Hu)^2 + (u - H^{-1}u')^2$

③ H模型代数距离: $e = Ah$ (见DLT求解H模型)

重复上述步骤得到内点数最多的模型

根据内点最多的模型得到内点匹配重新计算模型

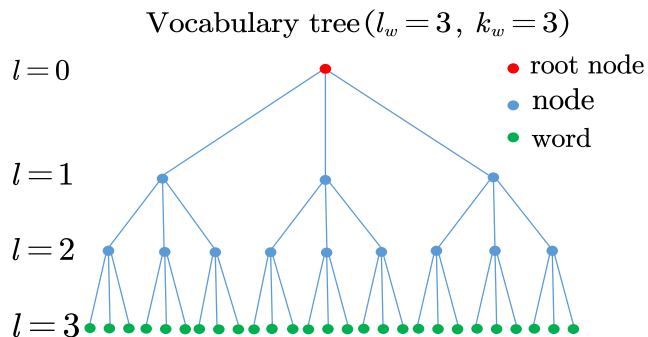
备注: F矩阵使用Sampson距离计算误差, 见《计算机视觉中的多视图几何》p60例3.2, p197 10.4.3小节

H矩阵使用对称转移误差, 见《计算机视觉中的多视图几何》p58 公式3.7

预备知识

- 1、3D-3D ICP与SE3/Sim3
- 2、3D-2D PnP相关算法
- 3、2D-3D Triangulation
- 4、2D-2D Homography/Fundamental
- 5、Bag-of-Words**
- 6、Orb 特征提取与匹配

Bag-of-Words



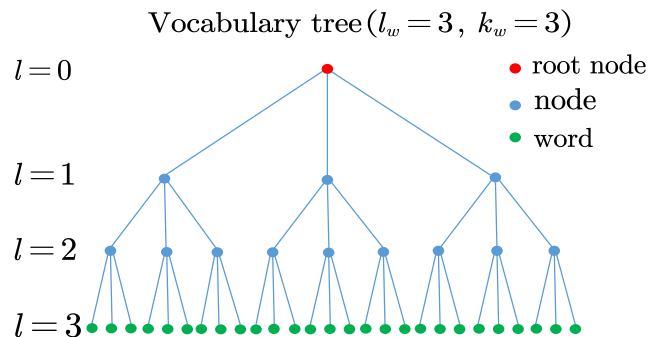
Bag - of - Words 语法树:

1. 通过聚类 (kmeans++) 的方式提前训练生成
2. root node、node、word都是描述子
3. 叶子节点称为words

Bag - of - Words 语法树作用:

1. 通过分层聚类的方式获得一个描述子到word的映射 $\rightarrow$ 描述子映射
(相比kd-tree, 分层语法树查找更快)
2. 可以将图像特征点描述子转换为图像描述子 (words) $\rightarrow$ 描述子压缩
(image $\rightarrow$ {features}_n $\rightarrow$ {words, scores}_m)
3. 属于同一节点的描述子是比较相似的描述子 $\rightarrow$ 描述子聚类
({features}_n $\rightarrow$ {{features}_{n1}, {features}_{n2}, {features}_{n3}...})
4. 生成“正向检索”“逆向检索”树

Bag-of-Words



正向检索文件:

vector < FeatureVector > directFile

$\nwarrow$ map < nodeId, vector < featureId > >

输入多张图像 + 语法树 + 转换层数 $\Rightarrow$ directFile:

$$\left\{ \begin{array}{l} \{ [\text{nodeId}_1, \{\text{featureIds}\}] [\text{nodeId}_2, \{\text{featureIds}\}] \cdots \}_{\text{image_1}} \\ \dots \\ \{ [\text{nodeId}_1, \{\text{featureIds}\}] [\text{nodeId}_2, \{\text{featureIds}\}] \cdots \}_{\text{image_n}} \end{array} \right.$$

逆向检索文件:

vector < IFRow > InvertFile

$\nwarrow$ list < IFPair >

$\nwarrow$ struct {imageId, wordValue}

map < wordId, wordValue > BowVector

输入多张图像 $\Rightarrow$ 每张图像转为BowVector $\Rightarrow$ InvertFile

$$\left\{ \begin{array}{l} \text{word}_1 \{ \text{IFPair}_{\text{image_5}}, \text{IFPair}_{\text{image_8}}, \text{IFPair}_{\text{image_10}} \cdots \} \\ \dots \\ \text{word}_n \{ \text{IFPair}_{\text{image_1}}, \text{IFPair}_{\text{image_3}}, \text{IFPair}_{\text{image_9}} \cdots \} \end{array} \right.$$

Bag-of-Words

语法树训练（聚类）原则：类间距离尽可能大，类内距离尽可能小
训练过程(kmeans ++):

- step1: 设定树的层数 l_w 、分支数 k_w 、迭代中止条件
(通常层数和分支数越大越好，一般取 $l_w = 6$, $k_w = 10$)
- step2: 随机设定初始聚类中心（和分支数一致）
- step3: 计算每个样本（描述子）到每个聚类中心的距离，
并选择距离最近的中心点并形成簇
- step4: 计算每个簇的中心点，并重复step3，直到中止条件
- step5: 计算words的权重(binary: 1, IDF: $\log N/N_i$)

在步骤2中，kmeans ++ 为了避免出现随机选择的中心点过近的问题，采用了优化策略：先随机选择一个中心点，其它所有样本距离中心点越远则被选作中心点的概率越大（转盘算法）

Bag-of-Words

计算两张图像words向量的距离:

$$\text{word value} = \begin{cases} \text{TF:} & 1 \\ \text{BINARY:} & 1 \\ \text{IDF:} & \log N/N_i \\ \text{TF-IDF:} & \log N/N_i \end{cases}$$

$$\begin{aligned} \|q - d\|^p &= \sum_i |q_i - d_i|^p \quad (\text{note: } q_i, d_i \text{ are normalized}) \\ &= \sum_{i|d_i=0} |q_i|^p + \sum_{i|q_i=0} |d_i|^p + \sum_{i|q_i \neq 0, d_i \neq 0} |q_i - d_i|^p \\ &= \|q\|^p + \|d\|^p + \sum_{i|q_i \neq 0, d_i \neq 0} (|q_i - d_i|^p - |q_i|^p - |d_i|^p) \\ &= 2 + \sum_{i|q_i \neq 0, d_i \neq 0} (|q_i - d_i|^p - |q_i|^p - |d_i|^p) \end{aligned}$$

$$\begin{aligned} L_1 \text{ score: } & 1 - \frac{1}{2} \cdot (2 + |q_i - d_i| - |q_i| - |d_i|) \\ &= -\frac{1}{2} \cdot (|q_i - d_i| - |q_i| - |d_i|) \\ L_2 \text{ score: } & 1 - \sqrt{\frac{1}{2} \cdot \left(2 - 2 \cdot \sum_{i|q_i \neq 0, d_i \neq 0} q_i d_i \right)} \\ &= 1 - \sqrt{1 - \sum_{i|q_i \neq 0, d_i \neq 0} q_i d_i} \\ \text{ChiSquare score: } & \frac{1}{2} \sum_{i|q_i \neq 0, d_i \neq 0} (q_i - d_i)^2 / (q_i + d_i) - q_i - d_i \\ &= 2 \cdot \sum_{i|q_i \neq 0, d_i \neq 0} q_i d_i / (q_i + d_i) \\ \text{KL score: } & \sum_{i|q_i \neq 0, d_i \neq 0} q_i \cdot \log(q_i/d_i) + \sum_{i|q_i \neq 0, d_i = 0} q_i \cdot \log(q_i) \\ \text{Bhattacharyya score: } & \sum_{i|q_i \neq 0, d_i \neq 0} \sqrt{q_i d_i} \end{aligned}$$

预备知识

- 1、3D-3D ICP与SE3/Sim3
- 2、3D-2D PnP相关算法
- 3、2D-3D Triangulation
- 4、2D-2D Homography/Fundamental
- 5、Bag-of-Words
- 6、Orb 特征提取与匹配

Orb特征提取与匹配

Orb算法中Fast特征点提取与Brief描述子计算比较直观这里不再赘述

正因为比较直观，对于OpenCV不熟悉的同学建议阅读这两个算法的源码来练习

这里主要介绍orbslam2中的两个技巧：

1. 如何避免图像一个区域内提取的特征点过于冗余或无特征点的情况

(1) 总共要提取 n 个特征点，那图像金字塔每一层提取多少个特征点

(2) 如何解决弱纹理区域提取不到特征点的问题

(3) 如何解决某些区域提取特征点密度过高的问题

2. 如何加速特征点的匹配以及提高匹配的正确性

(1) 图像金字塔与特征点尺度对应关系

(2) 根据旋转主方向、最优匹配与次优匹配距离比，减少错误匹配

(3) BoWs语法树缩小匹配范围，加速匹配

(4) 结合特征点深度与 $SE(3)$ 位姿，3D投影的方式缩小匹配范围

(5) 结合特征点深度与 $Sim(3)$ 位姿，3D投影的方式缩小匹配范围

(6) 根据极线约束减少错误匹配

Orb特征提取与匹配

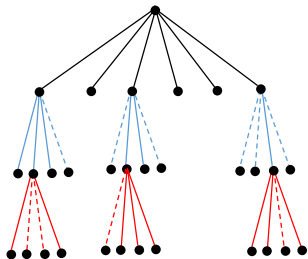
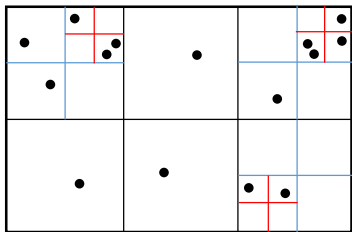
问题1.1: 第*i*层金字塔提取的特征数为 $N_i = N \cdot \frac{1-s}{1-s^i}$,

(*s*表示金字塔上下层图像相对尺度, *N*为总共要提取的特征点数)

问题1.2: a. 将图像划分为多个区域进行提取

b. 先用正常阈值进行FAST特征提取, 若提取不到特征点则降低阈值重新提取

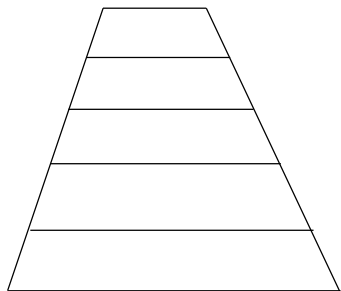
问题1.3: 采用四叉树对提取的特征点进行裁剪



Orb特征提取与匹配

问题2.1：图像金字塔与特征点尺度对应关系

Nearer



Farther

Level_ $(n - 1)$ $\rightarrow d_{min}$

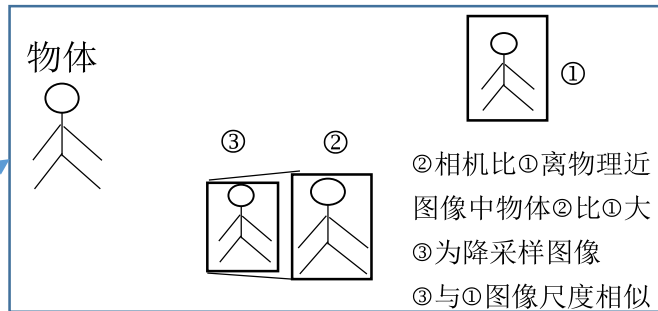
Level_ $m \rightarrow d$

Level_ $0 \rightarrow d_{max}$

$$\begin{aligned} \text{① } d/d_{min} &= s^{n-1-m} \Rightarrow d_{min} \\ \Rightarrow \text{② } d_{max}/d &= s^m \Rightarrow d_{max} \end{aligned}$$

$$\text{③ } m = \text{ceil}\left(\frac{d_{max}/d}{\log(s)}\right)$$

s表示金字塔上下层图像相对尺度



问题2.2：如何根据旋转主方向、最优匹配与次优匹配距离减少错误匹配

1. 可以统计整张图像的平均旋转量，如果匹配点描述子之间的相对旋转量和平均旋转量差异较大，则为错误匹配
2. 匹配过程中最优匹配的描述子距离满足阈值，但最优匹配与次优匹配距离相当，则判定匹配错误概率较大，匹配失败

Orb特征提取与匹配

问题2.3: 如何用BoWs语法树缩小匹配范围, 加速匹配

step1: 将匹配image1与被匹配图像image2的特征点都转为FeatureVector

step2: image1中每个特征点只匹配image2中与该特征点有相同nodeId的特征点

问题2.4/2.5 如何结合特征点深度与 $SE(3)$ / $Sim(3)$ 位姿, 3D投影的方式缩小匹配范围

step1: 特征点结合深度可以恢复出3D点

step2: 3D点通过 $SE(3)$ / $Sim(3)$ 可以将3D点从匹配坐标系变换到
被匹配坐标系, 并得到在被匹配图像上的投影点

step3: 可以通过3D点的深度预测被匹配特征点所在的金字塔层数

step4: image1每个特征点只匹配image2中投影点附近且金字层数
满足预测的特征点

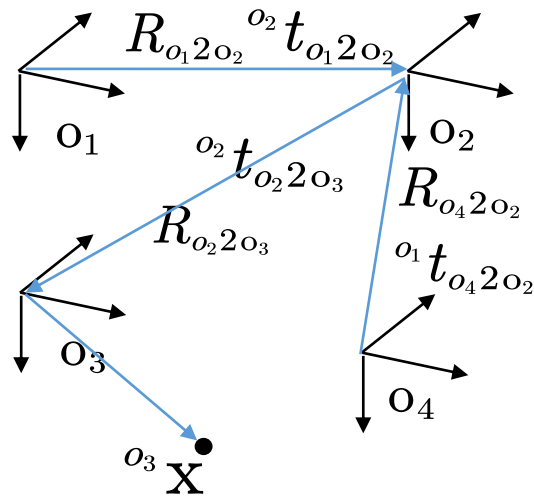
问题2.6 如何根据极线约束减少错误匹配

image1中每个特征点可以通过基础矩阵得到在image2上的极线,
如果被匹配特征点距离极线太远, 则跳过该匹配

补充1：坐标系变换

多坐标系之间坐标变换

如左图，求 x 在 o_4 坐标系下的表达？



关键点：

1. 搞清楚 R 和 t 的含义
2. 旋转可通过串性相乘的方式求取
3. 求取平移向量需要先将平移向量的坐标系统一到目标坐标系
(同一个旋转坐标系的向量相加才有正确的物理意义)

由上可推出：

1. 若 o_1 到 o_2 坐标系的旋转为 R ，平移为 t ，则把一个点从坐标系 o_1 变换到 o_2 坐标系的平移为 $-t$

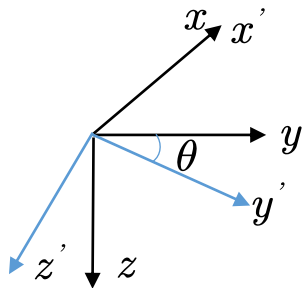
2. ${}^{o_2}\mathbf{x} = R_{o_1 \rightarrow o_2} \cdot {}^{o_1}\mathbf{x} + t$ ，则 t 表示 o_2 到 o_1 的向量在 o_2 坐标下的表达
3. ${}^{o_2}\mathbf{x} = R_{o_1 \rightarrow o_2} \cdot {}^{o_1}\mathbf{x} - t$ ，则 t 表示 o_1 到 o_2 的向量在 o_2 坐标下的表达
4. ${}^{o_2}\mathbf{x} = R_{o_1 \rightarrow o_2} \cdot ({}^{o_1}\mathbf{x} + t)$ ，则 t 表示 o_2 到 o_1 的向量在 o_1 坐标下的表达
5. ${}^{o_2}\mathbf{x} = R_{o_1 \rightarrow o_2} \cdot ({}^{o_1}\mathbf{x} - t)$ ，则 t 表示 o_1 到 o_2 的向量在 o_1 坐标下的表达

点坐标系变换直观

物理意义直观

补充1：坐标系变换

求2个坐标系之间的变换

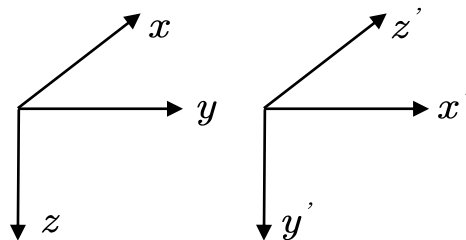


确定旋转正方向（一般右手方向为正方向）

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = R \begin{bmatrix} x \\ y \\ z \end{bmatrix} = R \cdot \left(\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} y + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} z \right)$$

原坐标系3个单位轴向量到变换后坐标系3个轴的投影：

$$\begin{cases} x' = x \\ y' = y \cos(\theta) + z \sin(\theta) \\ z' = -y \sin(\theta) + z \cos(\theta) \end{cases} \Rightarrow \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & \sin(\theta) \\ 0 & -\sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

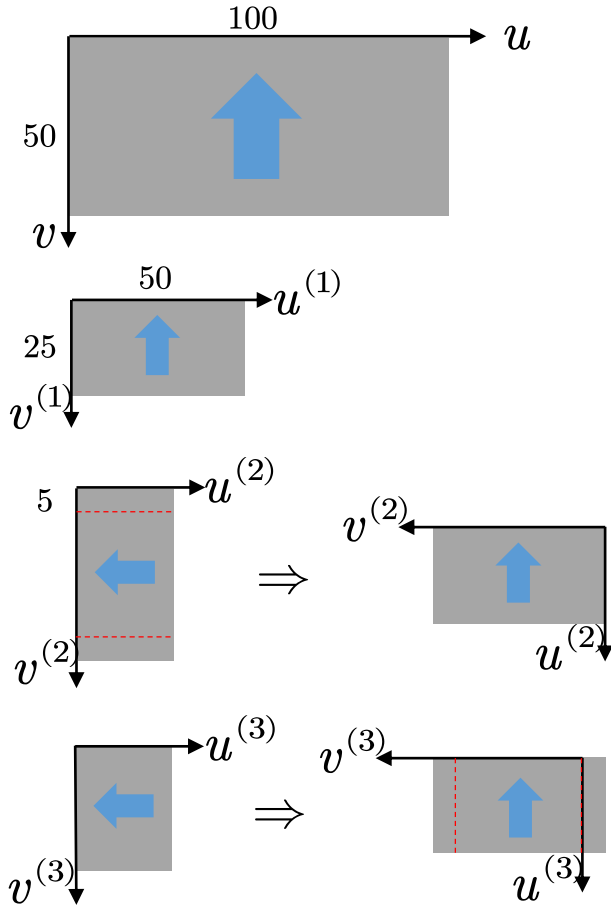


$$\begin{cases} x' = y \\ y' = z \\ z' = x \end{cases}$$

$$\Rightarrow \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

不需要用欧拉角转来转去，
通过此方法可以直接求解变换矩阵

补充1：坐标系变换



图像缩放与裁剪对应的变换

$$\begin{cases} u^{(1)} = \frac{1}{2}u \\ v^{(1)} = \frac{1}{2}v \end{cases}$$

$$\begin{cases} u^{(2)} = v^{(1)} = \frac{1}{2}v \\ v^{(2)} = (50 - 1) - u^{(1)} \\ \quad = (50 - 1) - \frac{1}{2}u \end{cases}$$

$$\begin{cases} u^{(3)} = u^{(2)} = \frac{1}{2}v \\ v^{(3)} = v^{(2)} - 5 = 44 - \frac{1}{2}u \end{cases}$$

$$u = \lambda \cdot A \cdot K \cdot R(x + t)$$

图像变换矩阵:

$$A = \begin{bmatrix} 0 & \frac{1}{2} & 0 \\ -\frac{1}{2} & 0 & 44 \\ 0 & 0 & 1 \end{bmatrix}$$

等效为:

$$u = \lambda \cdot K' \cdot R'(x + t)$$

$$\Rightarrow \begin{cases} R' = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} R \\ K' = \begin{bmatrix} \frac{1}{2}f_y & 1 & \frac{1}{2}v_0 \\ 0 & \frac{1}{2}f_x & 44 - \frac{1}{2}u_0 \\ 0 & 0 & 1 \end{bmatrix} \end{cases}$$

补充2 : 2D-2D 单应变换

归一化单映矩阵:

$$H = \frac{\hat{H}}{\lambda}$$

$$\Rightarrow \det(\hat{H}^T \hat{H} - \lambda \cdot I) = 0$$

$$\Rightarrow \lambda = \sqrt{\lambda_1^2 + \lambda_2^2 + \lambda_3^2}$$

在OpenCV: $\lambda = \lambda_2$

归一化 H 矩阵后, 存在:

$$S = H^T H - I = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{12} & s_{22} & s_{23} \\ s_{13} & s_{23} & s_{33} \end{bmatrix}$$

$$\det(S^-) = s_{11}s_{22}s_{33} - s_{11}s_{23}^2 - s_{22}s_{13}^2 - s_{33}s_{12}^2 + 2s_{12}s_{13}s_{23} = 0$$

Homography模型分解

$$H = R_{o_2o_1} + \frac{1}{d} \cdot t_{o_2o_1} \cdot n^T, \text{ 简写为 } H = R + t \cdot n^T$$

$$S = H^T H - I$$

$$= (R^T + n \cdot t^T)(R + t \cdot n^T) - I$$

$$= R^T \cdot t \cdot n^T + n \cdot t^T \cdot R + n \cdot t^T \cdot t \cdot n^T$$

$$\text{令: } \begin{cases} \mathbf{x} = \frac{R^T \cdot t}{\|R^T \cdot t\|} = \frac{R^T \cdot t}{\|t\|} = [x_1, x_2, x_3]^T & (1) \\ \mathbf{y} = \|R^T \cdot t\| \cdot n = \|t\| \cdot n = [y_1, y_2, y_3]^T & (2) \end{cases}, \text{ 则:}$$

$$S = \mathbf{xy}^T + \mathbf{yx}^T + \mathbf{yy}^T$$

$$\Rightarrow \begin{bmatrix} y_1^2 + 2y_1x_1 & y_2x_1 + y_1x_2 + y_1y_2 & y_3x_1 + y_1x_3 + y_1y_3 \\ - & y_2^2 + 2y_2x_2 & y_3x_2 + y_2x_3 + y_2y_3 \\ - & - & y_3^2 + 2y_3x_3 \end{bmatrix} = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{12} & s_{22} & s_{23} \\ s_{13} & s_{23} & s_{33} \end{bmatrix}$$

$$\Rightarrow \begin{cases} y_1^2 + 2y_1x_1 = s_{11} \\ y_2^2 + 2y_2x_2 = s_{22} \\ y_3^2 + 2y_3x_3 = s_{33} \\ y_2x_1 + y_1x_2 + y_1y_2 = s_{12} \\ y_3x_1 + y_1x_3 + y_1y_3 = s_{13} \\ y_3x_2 + y_2x_3 + y_2y_3 = s_{23} \end{cases}$$

Malis分解方法

2D-2D 单应变换

Homography模型分解

Malis分解方法

用前三个式子计算出 x_1 、 x_2 、 x_3 带入后三个式子从而消掉 x_1 、 x_2 、 x_3

$$\begin{cases} y_1^2 + 2y_1x_1 = s_{11} \\ y_2^2 + 2y_2x_2 = s_{22} \\ y_3^2 + 2y_3x_3 = s_{33} \\ y_2x_1 + y_1x_2 + y_1y_2 = s_{12} \\ y_3x_1 + y_1x_3 + y_1y_3 = s_{13} \\ y_3x_2 + y_2x_3 + y_2y_3 = s_{23} \end{cases} \Rightarrow \begin{cases} x_1 = \frac{s_{11} - y_1^2}{2y_1} \quad (3) \\ x_2 = \frac{s_{22} - y_2^2}{2y_2} \quad (4) \\ x_3 = \frac{s_{33} - y_3^2}{2y_3} \quad (5) \\ s_{22}y_1^2 - 2s_{12}y_1y_2 + s_{11}y_2^2 = 0 \\ s_{11}y_3^2 - 2s_{13}y_1y_3 + s_{33}y_1^2 = 0 \\ s_{33}y_2^2 - 2s_{23}y_2y_3 + s_{22}y_3^2 = 0 \end{cases}$$

$$\text{令: } \begin{cases} z_1 = \frac{y_1}{y_2} \quad (6) \\ z_2 = \frac{y_1}{y_3} \quad (7) \\ z_3 = \frac{y_3}{y_2} \quad (8) \end{cases} \Rightarrow \begin{cases} s_{22}z_1^2 - 2s_{12}z_1 + s_{11} = 0 \\ s_{33}z_2^2 - 2s_{13}z_2 + s_{11} = 0 \\ s_{22}z_3^2 - 2s_{23}z_3 + s_{33} = 0 \end{cases}$$

$$\Rightarrow \begin{cases} z_1 = \alpha_1 \pm \beta_1, \quad \alpha_1 = \frac{s_{12}}{s_{22}}, \quad \beta_1 = \frac{\sqrt{s_{12}^2 - s_{11}s_{22}}}{s_{22}} \quad (9) \\ z_2 = \alpha_2 \pm \beta_2, \quad \alpha_2 = \frac{s_{13}}{s_{33}}, \quad \beta_2 = \frac{\sqrt{s_{13}^2 - s_{11}s_{33}}}{s_{33}} \quad (10) \\ z_3 = \alpha_3 \pm \beta_3, \quad \alpha_3 = \frac{s_{12}}{s_{22}}, \quad \beta_3 = \frac{\sqrt{s_{23}^2 - s_{22}s_{33}}}{s_{22}} \quad (11) \end{cases}$$

如果 $s_{22} = 0$, 则可以令:

$$\begin{cases} z_1 = \frac{y_2}{y_1} \\ z_2 = \frac{y_3}{y_1} \\ z_3 = \frac{y_2}{y_3} \end{cases}, \text{ 并重新推导整个过程}$$

2D-2D 单应变换

Homography模型分解

Malis分解方法

(1) $\Rightarrow x_1^2 + x_2^2 + x_3^2 = 1$, 根据(3)(4)(5):

$$\Rightarrow x_1^2 + x_2^2 + x_3^2 = \frac{(s_{11} - y_1^2)^2}{4y_1^2} + \frac{(s_{22} - y_2^2)^2}{4y_2^2} + \frac{(s_{33} - y_3^2)^2}{4y_3^2} = 1$$

令: $w = y_2^2$, 则(6)(7)(8) $\Rightarrow y_1^2 = z_1^2 y_2^2$, $y_3^2 = z_3^2 y_2^2$

$$\Rightarrow (s_{22} - w)^2 + \frac{(s_{11} - wz_1^2)^2}{z_1^2} + \frac{(s_{33} - wz_3^2)^2}{z_3^2} - 4w = 0$$

$$\Rightarrow aw^2 - 2bw + c = 0$$

$$\begin{cases} a = 1 + z_1^2 + z_3^2 \\ b = 2 + s_{11} + s_{22} + s_{33} = 2 + \text{trace}(S) \\ c = s_{22}^2 + \frac{s_{11}^2}{z_1^2} + \frac{s_{33}^2}{z_3^2} \end{cases}$$

$$w = y_2^2 = \frac{b \pm \sqrt{b^2 - 4ac}}{2a} \Rightarrow y_2 = \pm \sqrt{w} \quad (12)$$

$$(6)(7)(8) \Rightarrow y = \begin{bmatrix} z_1 \\ 0 \\ z_3 \end{bmatrix} \cdot y_2 \quad (13) \quad (y \text{ 的计算与 } z_2 \text{ 无关})$$

$$y = \begin{bmatrix} z_1 \\ 0 \\ z_3 \end{bmatrix} \cdot y_2,$$

z_1 和 z_3 各有2种可能, y_2 有2种可能, 共8个可能的解
其中4个解表示平面在相机前方, 另外4个解在相机后方
接下来介绍如何排除在相机后方的4个解

2D-2D 单应变换

Homography模型分解

Malis分解方法

由Homography示意图 $\Rightarrow 1 + n^T R^T t > 0$

$$\Rightarrow 1 + n^T R^T t = 1 + y^T x > 0$$

$$\Rightarrow y^T x = [y_1 \ y_2 \ y_3] \begin{bmatrix} \frac{s_{11} - y_1^2}{2y_1} \\ \frac{s_{22} - y_2^2}{2y_2} \\ \frac{s_{33} - y_3^2}{2y_3} \end{bmatrix} \geq -1$$

$$\Rightarrow \frac{s_{11} - y_1^2}{2} + \frac{s_{22} - y_2^2}{2} + \frac{s_{33} - y_3^2}{2} \geq -1$$

$$\Rightarrow s_{11} + s_{22} + s_{33} + 2 \geq (y_1^2 + y_2^2 + y_3^2)$$

$$\Rightarrow b \geq w(1 + z_1^2 + z_3^2) = wa$$

$$(12) \Rightarrow \begin{cases} w = \frac{b + \sqrt{b^2 - ac}}{a} & \text{不成立} \\ w = \frac{b - \sqrt{b^2 - ac}}{a} & \text{成立 (14)} \end{cases}$$

算法总结:

step1: (9) (10) (11) $\Rightarrow z_1, z_3$

step2: (14) $\Rightarrow w$

step3: (13) $\Rightarrow y = [y_1 \ y_2 \ y_3]^T$

step4: $n = y/||y|| \Rightarrow n$

step5: (3) (4) (5) $\Rightarrow x = [x_1 \ x_2 \ x_3]^T$

step6: 参考坐标系下 ${}^o t_{o_2o_1} = R^T t = x$

step7: $R = H \cdot (I + xy^T)^{-1}$

2D-2D 单应变换

Homography模型分解

Malis分解方法

上一页PPT的算法流程可进一步简化为:

$$\text{step1: } M_{S_{11}} = - \begin{vmatrix} s_{22} & s_{23} \\ s_{23} & s_{33} \end{vmatrix} = s_{23}^2 - s_{22}s_{33}$$

$$M_{S_{22}} = - \begin{vmatrix} s_{11} & s_{13} \\ s_{13} & s_{33} \end{vmatrix} = s_{13}^2 - s_{11}s_{33}$$

$$M_{S_{33}} = - \begin{vmatrix} s_{11} & s_{12} \\ s_{12} & s_{22} \end{vmatrix} = s_{12}^2 - s_{11}s_{22}$$

$$\text{step2: } s_{max} = \max\{s_{11}, s_{22}, s_{33}\}$$

$$\text{当 } s_{max} = s_{11} \text{ 时, } n_a = \begin{bmatrix} s_{11} \\ s_{12} + \sqrt{M_{S_{33}}} \\ s_{13} + \epsilon_{23} \cdot \sqrt{M_{S_{33}}} \end{bmatrix}, n_b = \begin{bmatrix} s_{11} \\ s_{12} - \sqrt{M_{S_{33}}} \\ s_{13} - \epsilon_{23} \cdot \sqrt{M_{S_{33}}} \end{bmatrix}$$

$$\text{当 } s_{max} = s_{22} \text{ 时, } n_a = \begin{bmatrix} s_{12} + \sqrt{M_{S_{33}}} \\ s_{22} \\ s_{23} - \epsilon_{13} \cdot \sqrt{M_{S_{11}}} \end{bmatrix}, n_b = \begin{bmatrix} s_{12} - \sqrt{M_{S_{33}}} \\ s_{22} \\ s_{23} + \epsilon_{13} \cdot \sqrt{M_{S_{11}}} \end{bmatrix}$$

$$\text{当 } s_{max} = s_{33} \text{ 时, } n_a = \begin{bmatrix} s_{12} + \epsilon_{12} \cdot \sqrt{M_{S_{22}}} \\ s_{23} + \sqrt{M_{S_{11}}} \\ s_{33} \end{bmatrix}, n_b = \begin{bmatrix} s_{12} + \epsilon_{12} \cdot \sqrt{M_{S_{22}}} \\ s_{23} + \sqrt{M_{S_{11}}} \\ s_{33} \end{bmatrix}$$

$$\text{其中, } \epsilon_{ij} = \text{sign}(M_{S_{ij}}) = \begin{cases} 1 & , M_{S_{ij}} \geq 0 \\ -1 & , M_{S_{ij}} < 0 \end{cases}$$

$$\begin{aligned} \text{step3: } \nu &= \sqrt{2[(1 + \text{trace}(S))^2 + 1 - \text{trace}(S^2)]} \\ &= 2\sqrt{1 + \text{trace}(S) - M_{S_{11}} - M_{S_{22}} - M_{S_{33}}} \end{aligned}$$

$$ii = \max_{ii} \{s_{11}, s_{22}, s_{33}\}$$

$$\epsilon_{ii} = \text{sign}(s_{ii})$$

$$\rho^2 = 2 + \text{trace}(S) + \nu$$

$$||t||^2 = 2 + \text{trace}(S) - \nu$$

$$\text{step4: } \begin{cases} t_a(s_{ii}) = \frac{||t||}{2} [\epsilon_{ii} \cdot \rho \cdot n_b(s_{ii}) - ||t|| \cdot n_a(s_{ii})] \\ \Rightarrow \begin{cases} (R_a, t_a, n_a) \\ (R_a, -t_a, -n_a) \end{cases} \\ t_b(s_{ii}) = \frac{||t||}{2} [\epsilon_{ii} \cdot \rho \cdot n_a(s_{ii}) - ||t|| \cdot n_b(s_{ii})] \\ \Rightarrow \begin{cases} (R_b, t_b, n_b) \\ (R_b, -t_b, -n_b) \end{cases} \end{cases}$$

$$\text{step5: } R = H \cdot \left(I - \frac{2}{\nu} \cdot t \cdot n^T \right), {}^o t = R \cdot t$$

2D-2D 单应变换

Homography模型分解

Zhang SVD 方法

$$H^T H = V \Lambda^2 V^T, \Lambda = \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \lambda_3 \end{bmatrix} V = [v_1 \ v_2 \ v_3]$$

$$\text{令: } {}^\circ t = R^T t$$

$$\begin{cases} ||{}^\circ t|| = \lambda_1 - \lambda_3 \\ n^T \cdot {}^\circ t = \lambda_1 \lambda_3 - 1 \\ v_1 \propto v_1' = \zeta_1 \cdot {}^\circ t + n = ||v_1'|| \cdot v_1 \\ v_2 \propto v_2' = {}^\circ t \times n = ||v_2'|| \cdot v_2 \\ v_3 \propto v_3' = \zeta_3 \cdot {}^\circ t + n = ||v_3'|| \cdot v_3 \end{cases}$$

$$\text{其中: } \begin{cases} \zeta_1 = \frac{1}{2\lambda_1\lambda_3} \left(-1 + \sqrt{1 + 4 \frac{\lambda_1\lambda_3}{(\lambda_1 - \lambda_3)^2}} \right) \\ \zeta_3 = \frac{1}{2\lambda_1\lambda_3} \left(-1 - \sqrt{1 + 4 \frac{\lambda_1\lambda_3}{(\lambda_1 - \lambda_3)^2}} \right) \\ ||v_1'|| = \sqrt{\zeta_1^2 (\lambda_1 - \lambda_3)^2 + 2\zeta_1 (\lambda_1\lambda_3 - 1) + 1} \\ ||v_3'|| = \sqrt{\zeta_3^2 (\lambda_1 - \lambda_3)^2 + 2\zeta_3 (\lambda_1\lambda_3 - 1) + 1} \end{cases}$$

$$\begin{cases} {}^\circ t = \pm \frac{v_1' - v_3'}{\zeta_1 - \zeta_3}, n = \pm \frac{\zeta_1 v_3' - \zeta_3 v_1'}{\zeta_1 - \zeta_3} \\ {}^\circ t = \pm \frac{v_1' + v_3'}{\zeta_1 - \zeta_3}, n = \pm \frac{\zeta_1 v_3' + \zeta_3 v_1'}{\zeta_1 - \zeta_3} \end{cases}$$

平面在同一侧检查: $1 + n^T \cdot R^T \cdot t > 0$

补充3：H矩阵和E矩阵的关系

(1) 如果匹配的像素点满足(基于平面诱导的) H 矩阵约束，则也满足 F 矩阵约束

(2) e' 是相机 o 在相机 o' 图像上的极点， u' 为 o' 图像上一个像素点，则：

$$\begin{aligned}\text{极线 } l' &= e' \times u' \\ &= [e']_{\times} u' \\ &= [e']_{\times} H u \\ &= F u\end{aligned}$$

$\Rightarrow F = [e']_{\times} H = H^{-T} [e']_{\times}$ ，即 H 矩阵和一个极点即可计算出 F 矩阵

(3) $u'^T F u = 0$

$$\Rightarrow (H u)^T F u = 0$$

$$\Rightarrow u^T H^T F u = 0$$

$$\Rightarrow H^T F + F^T H = 0$$

(4) $\det(F) = 0$

(5) $EE^T E = \frac{1}{2} \text{tr}(EE^T) E$ ，见备注

(6) 当只有平移时， $R = I$

$$u' = \frac{1}{z} K(Rx + t) = \frac{1}{z} K(x + t) = u + \frac{1}{z} Kt$$

备注： E 矩阵自身约束推导可见“*Theory of Reconstruction from Image Motion*” Theorem 2.12

补充4：5点法求解E矩阵

先列出两个理论公式：

$$\begin{cases} EE^T E = \frac{1}{2} \text{tr}(EE^T) E & (1) \\ \det(F) = \det(K_2^{-T} EK_1^{-1}) = 0 & (2) \end{cases}$$

$\bar{x}'^T E \bar{x} = 0$ ，根据DLT方法，可写为 $Ae = 0$

其中： $A_i = [\bar{x}'_x \bar{x}_x \quad \bar{x}'_x \bar{x}_y \quad \bar{x}'_y \bar{x}_x \quad \bar{x}'_y \bar{x}_y \quad \bar{x}'_y \bar{x}_y \quad \bar{x}'_y \bar{x}_x \quad \bar{x}_x \quad \bar{x}_y \quad 1]_i$

$$e = [e_0 \quad e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \quad e_6 \quad e_7 \quad e_8]^T$$

根据5对匹配点，可以得到5*9的A矩阵， $A \stackrel{\text{SVD}}{=} U \Sigma V^T$

通过最小4个特征点对应的特征向量 X_v, Y_v, Z_v, W_v 可扩展出A的右零空间

将 X_v, Y_v, Z_v, W_v 写成3*3的矩阵： X, Y, Z, W

则 $E = xX + yY + zZ + wW$ ，由于E是欠尺度的，故令 $w = 1$

带入上式(1)中，可以得到9个方程（带入过程最好借助Matlab）

带入上式(2)中，可以得到1个方程。共10个方程。

备注：1. “Five — Point Motion Estimation Made Easy”

2. “An Efficient Solution to the Five — Point Relative Pose Problem”

3. “OpenCV源代码：five — point.cpp” 4. “<https://nghiaho.com/?p=1675>”

补充4：5点法求解E矩阵

这十个方程的形式为： $MX=0$ ， M 为常值矩阵， X 为参数相关向量：

$$[x^3, y^3, x^2y, xy^2, x^2, y^2, xy, x, y, 1, z^3, xyz, x^2z, y^2z, xz^2, yz^2, z^2, xz, yz, z]^T$$

针对 $MX=0$ ，通过引入隐变量的方法求解 x, y, z

隐变量法举例： $a_ix + b_iy + c_iz + d_i = 0$

$$\text{即：} [a, b, c, d]_i \begin{bmatrix} x \\ y \\ xz \\ 1 \end{bmatrix} = 0$$

如果把 z 当做隐变量，则：

$$[a + cz, b, d]_i \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = 0$$

$\det(C(z)) = 0$ 可以得到一个关于 z 的10阶方程，
得到多个实数解（最多10个解）

对于每一个 z_i ，可计算出 $C(z)_i$

现在，根据 $CX=0$ ，求解出 X 中的 x 与 y ：

对 C 进行SVD分解，得到最小奇异值对应的特征向量 X^*

由于 $X = [x^3, y^3, x^2y, xy^2, x^2, y^2, xy, x, y, 1]^T$ ，得到：

$$x = X^*[7]/X^*[9], \quad y = X^*[8]/X^*[9]$$

$$E^* = xX + yY + zZ + wW$$

将 z 作为隐变量，则 $MX=0$ 变为： $C(z)X(x,y)=0$

$$X(x, y) = [x^3, y^3, x^2y, xy^2, x^2, y^2, xy, x, y, 1]^T$$

对于齐次方程组，得到非平凡解（非零解）的条件是 $\det(C(z)) = 0$

补充5：优化的方法求F矩阵

优化 $\min ||x'Fx||$

$$F \stackrel{\text{SVD}}{=} U \Sigma V^T$$

U 和 V 为正交矩阵，由于 F 的秩为2， $\Sigma = \text{diag}(1, s, 0)$

$$\begin{cases} U' = \Delta R_1 U \\ V' = \Delta R_2 V \\ s' = s + \Delta s \end{cases}$$

其中 ΔR_1 和 ΔR_2 可用欧拉角或李代数参数化，共7个参数，7个自由度

补充6：齐次最小二乘解

找最优 x 使得 $Ax=0$ ，建立优化模型：

$$\min \|Ax\|$$

s.t. $\|x\|=1$ (如果 x^* 为最优解，则 λx^* 也为最优解，故加此约束)

对 A 进行SVD分解： $A = U \overset{\text{SVD}}{\Sigma} V^T$ ，此时优化问题：

$$\min \|Ax\|$$

$$= \min \|U \Sigma V^T x\|$$

$$= \min \|\Sigma V^T x\|$$

令 $y = V^T x$ ，由于 $\|x\|=1 \Rightarrow \|y\|=1$ ，此时优化问题：

$$\min \|\Sigma y\|, \text{ s.t. } \|y\|=1$$

由于 Σ 为对角矩阵，且数值降序排列，故：

取 $y = [0, 0 \dots, 1]^T$ 时得到最优解

由于 $x = Vy$ ，故 x^* 等于 V 的最后一列

即：齐次最小二乘解为矩阵最小奇异值对应的特征向量

$m \times n$ 大小的矩阵SVD分解的复杂度为：

$$4m^2n + 8mn^2 + 9n^3$$

当只求 V 和 Σ 时，复杂度为：

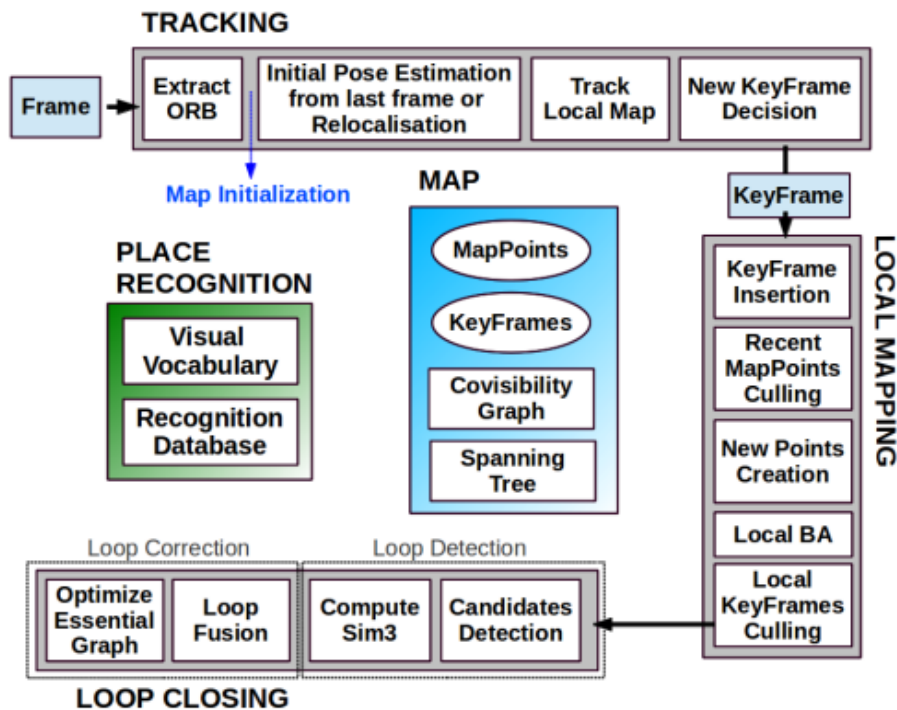
$$4m^2n + 8n^3$$

故：求解齐次最小二乘时可以不求解 U 来减少运算量

- 预备知识
- **视觉VO与重定位**
- 局部优化
- 全局闭环

整体软件结构

主要线程:



主要逻辑文件:

System.cpp --> class System

Initializer.cpp --> class Initializer

Tracking.cpp --> class Tracking

LocalMapping.cpp --> class LocalMapping

LoopClosing.cpp --> class LocalClosing

Viewer.cpp --> class Viewer

主要类:

class Frame

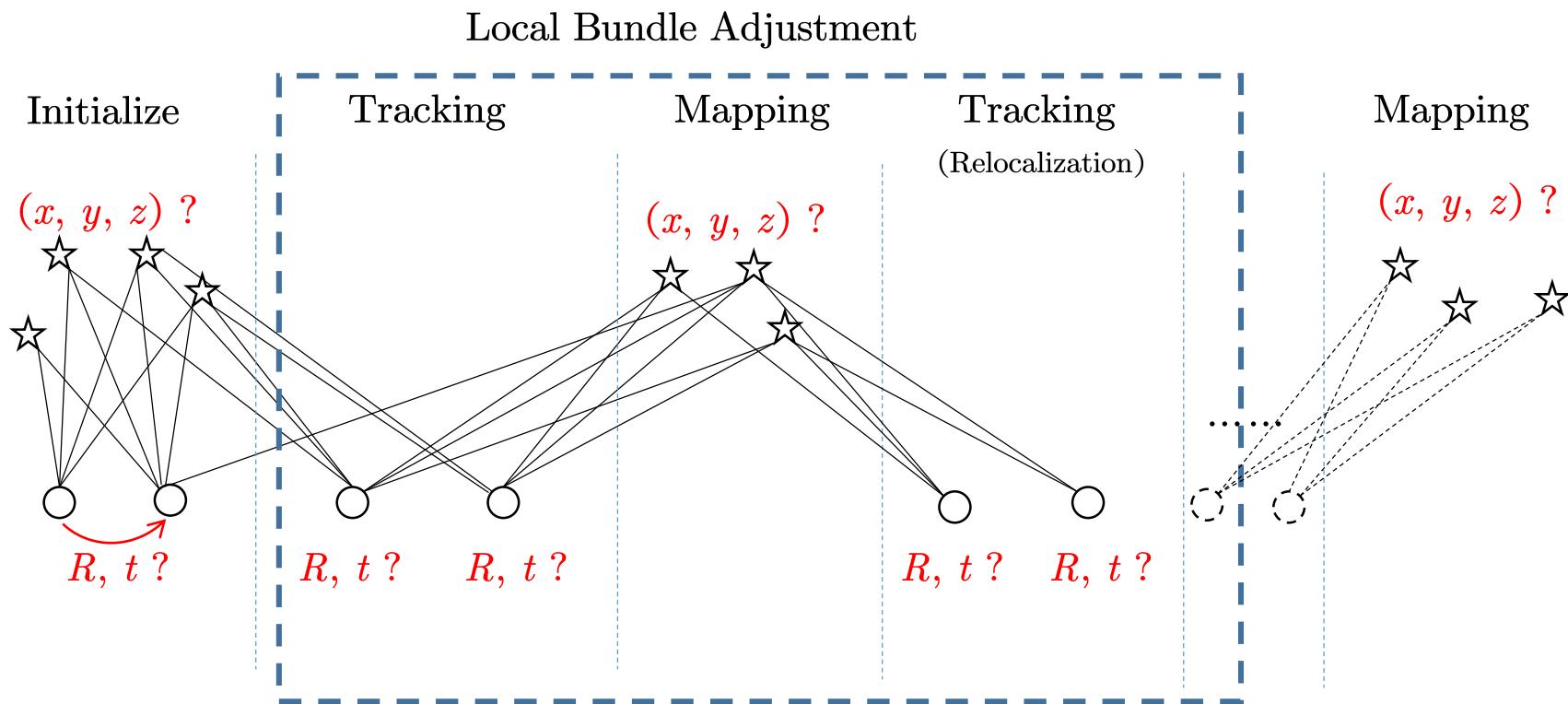
class KeyFrame

class KeyFrameDatabase

class Map

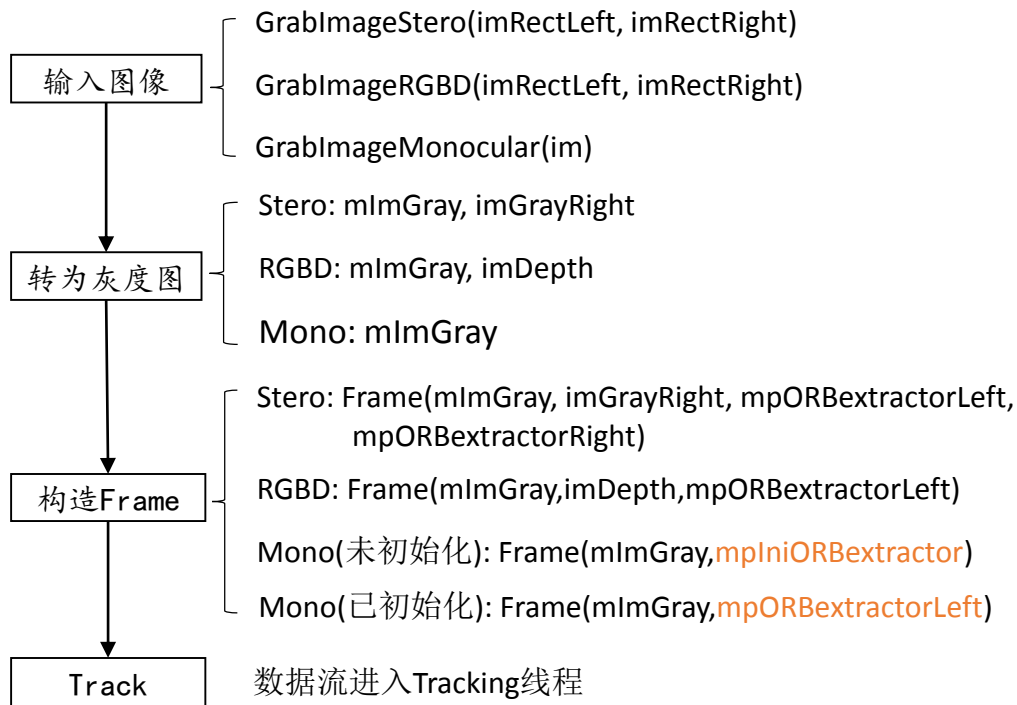
class Optimizer

最简单的VO流程



视觉跟踪与建图

主线程System入口



双目构造Frame需要左右相机找到匹配特征点:

step1: 为左目每个特征点建立一个带状区域搜索表, 限定搜索区域。(已提前极线矫正)

step2: 通过描述子进行特征点匹配, 得到每个特征最佳匹配点scaledR0

step3: 通过SAD滑窗得到匹配修正量bestincR

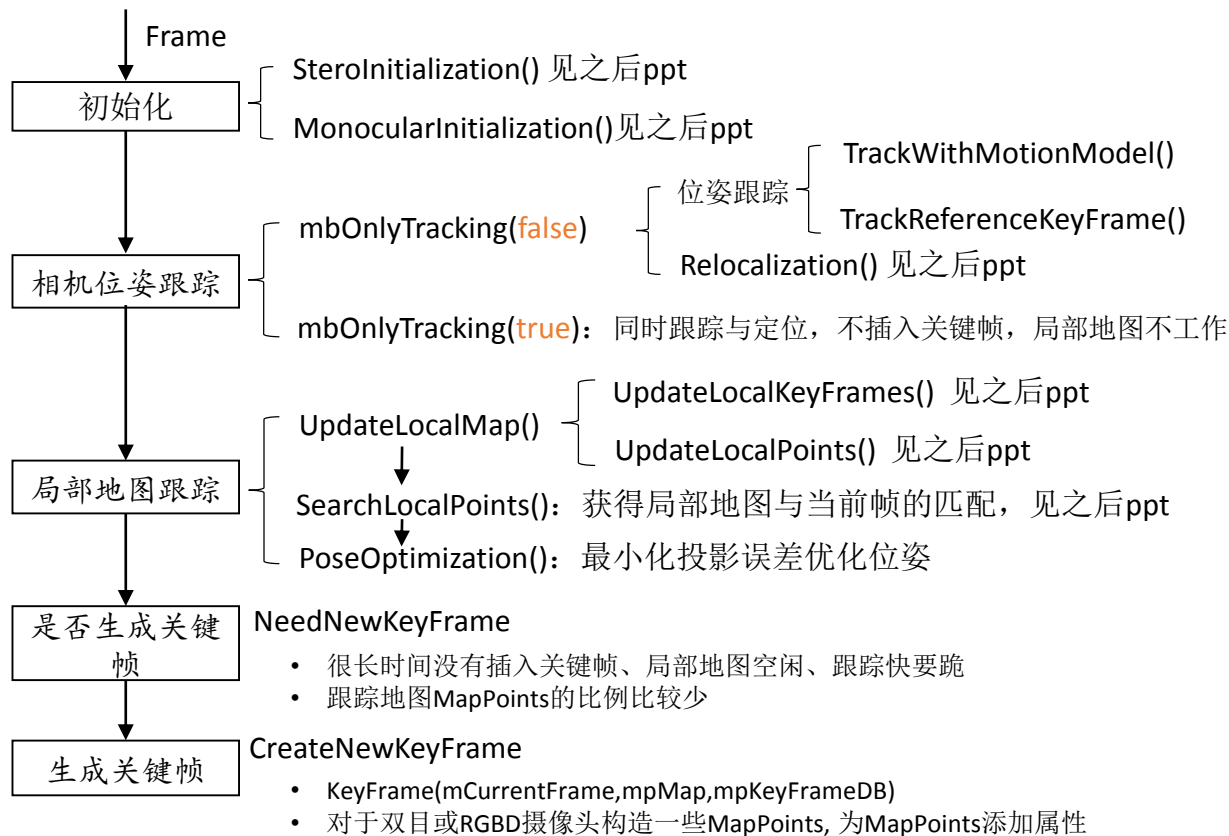
step4: $(bestincR, dist)$ $(bestincR - 1, dist)$ $(bestincR + 1, dist)$ 三个点拟合抛物线, 得到亚像素修正量deltaR

step5: 最终匹配点位置为:
 $scaleduR0 + bestincR + deltaR$

注: `mpIniORBextractor`相比`mpORBextractorLeft`提取的特征点多一倍

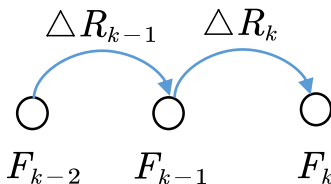
视觉跟踪与建图

Tracking线程



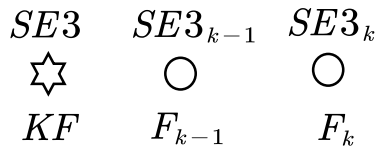
注: **mbOnlyTracking**默认为false, 用户可通过运行界面选择仅跟踪定位模式

TrackWithMotionModel



恒速模型: 假设 $\Delta R_k \approx \Delta R_{k-1}$

TrackReferenceKeyframe



跟踪参考帧模型: $SE3_k \approx SE3_{KF}$

视觉跟踪与建图

Tracking线程

对于双目或RGBD，可直接恢复一些3D点。
因此单帧即可创建Mappoints，KeyFrame等等

对于单目，通过时序上两帧观测恢复H和F模型、分解、以及三角化恢复3D点，并通过得分的方式选择一个更好的模型结果进行初始化

Initialization

Fundamental模型评分

$$\text{scoreF} = \sum_{i=0}^N \rho(T_F - \|x'Fx\|^2/\sigma^2)$$

$$\rho(x) = \begin{cases} 0 & x \leq 0 \\ x & \text{else} \end{cases} \quad T_F = 3.84$$

Homography模型评分

$$\text{scoreH} = \sum_{i=0}^N \rho(T_H - \|x' - Hx\|^2/\sigma^2)$$

$$\rho(x) = \begin{cases} 0 & x \leq 0 \\ x & \text{else} \end{cases} \quad T_H = 5.99$$

Fundamental模型与Homography模型选择

$$R_H = \frac{s_H}{s_H + s_F}, R_H \geq 0.45$$

UpdateLocalKeyFrames

能观测到当前帧mCurrentFrame的Mappoints的关键帧--->keyframeCounter

除去无效的关键帧--->.mvpLocalKeyFrames

- .mvpLocalKeyFrames中与其它（共视）连接最多的关键帧作为参考帧mpReferenceKF

共视关键帧也添加进来

- .mvpLocalKeyFrames中每一帧共视比较好的10帧也添加进来
- 自己的子关键帧
- 自己的父关键帧

UpdateLocalPoints

收集.mvpLocalKeyFrames中每帧关键帧的Mappoints --->mvLocalMapPoints

SearchLocalPoints

标记当前帧的Mappoints，这些Mappoints不同搜索匹配

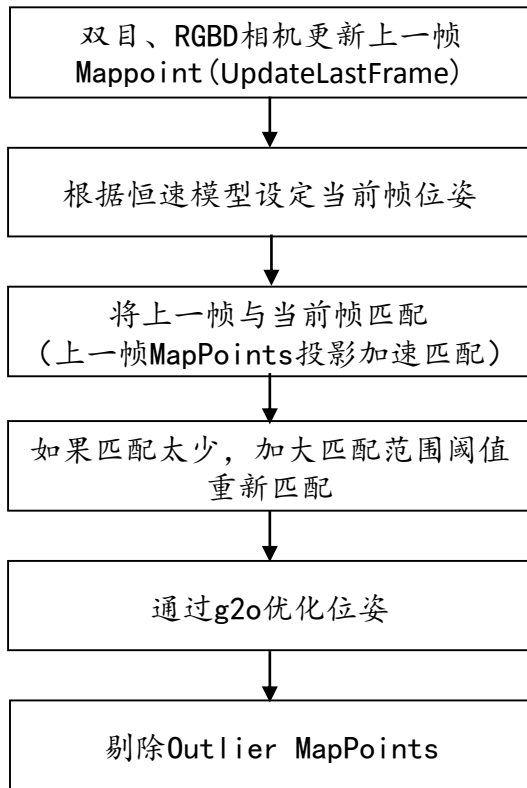
遍历mvpLocalMappoints判断是否在mCurrentFrame视野内

- 如果在视野内，则该mappoint被观测的统计值加1

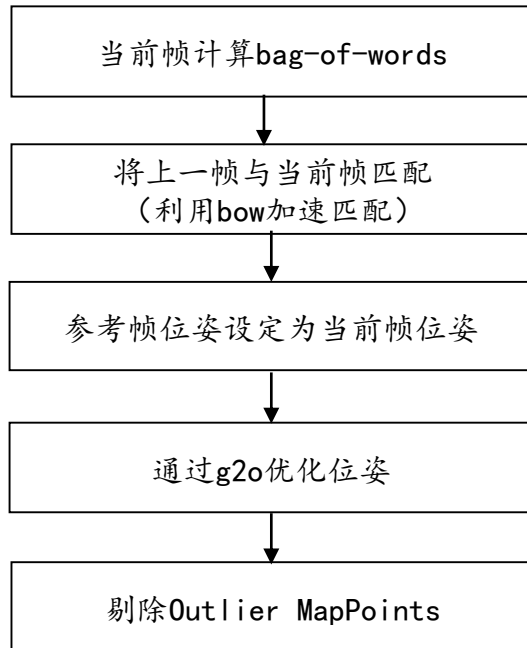
当前帧mCurrentFrame通过投影匹配mvpLocalMappoints

- 调用SearchByProject

TrackWithMotionModel



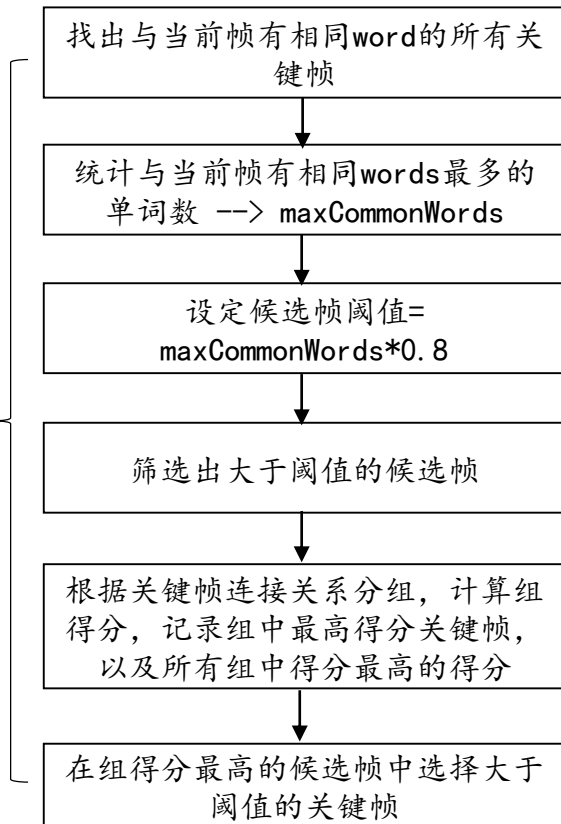
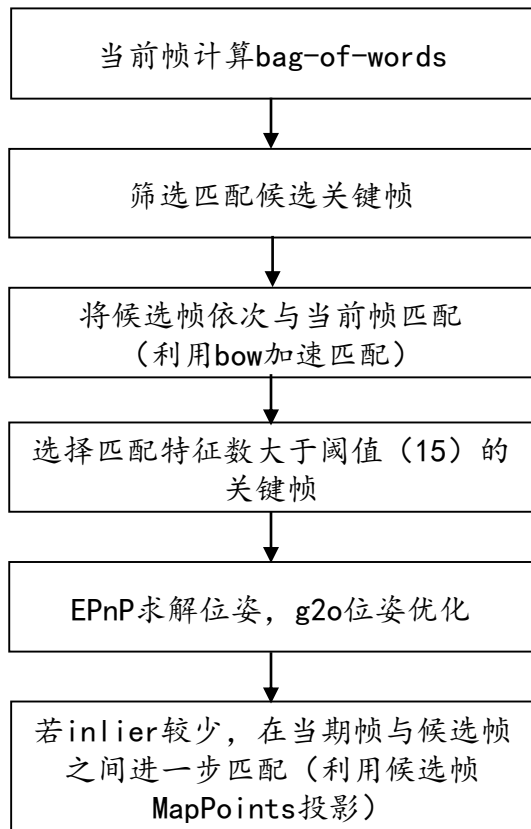
TrackWithReferenceKeyframe



视觉跟踪与建图

Tracking线程

Relocalization



```
lAccScoreAndMatch =  
{(accScore, pBestKF)}  
bestAccScore = max{accScore}
```

- 预备知识
- 视觉VO与重定位
- **局部优化**
- 全局闭环

局部优化



局部优化

UpdateConnections

1. 取出当前关键帧mpCurrentKeyFrame所有的MapPoints--->mvpMapPoints
2. 取出能观测到这些MapPoints的关键帧--->observations
3. 统计每个关键帧观测到这些MapPoints的个数--->Kfcounter<KeyFrame*, int>
4. 建立共视有向连接图
 - 如果这些关键帧中观测到MapPoints的个数大于阈值，则与当前关键帧建立连接 ---> vPairs
 - 得到共视关系最好的关键帧 --->pKFmax
 - 如果这些关键帧中观测到MapPoints的个数均小于阈值，则将pKFmax与当前帧建立连接---> vPairs
 - 对vPairs按照共视的程度进行排序
 - vPairs中共视关系最强的关键帧作为mpCurrentKeyFrame的父关键帧，同时mpCurrentKeyFrame作为该帧的子关键帧

局部优化

MapPointCulling

作用：剔除mlpRecentAddedMapPoints中不好的MapPoints

mlpRecentAddedMapPoints来自于：

- 对于双目会在创建关键帧时新建一些MapPoints
- 通过CreateNewMapPoints函数，关键帧之间三角化恢复一些MapPoints

剔除MapPoint的标准为（观测一致性，从被创建开始，未被连续可靠观测到）：

- $mnFound / mnVisible < 25\%$
- 从MapPoint被创建开始，到现在已经过了不小于2帧关键帧，但是该MaPoints被观测到的关键帧数小于阈值（单目为2，双目为3）
- 从MapPoint被创建开始，到现在已经超过帧关键帧

局部优化

CreateNewMapPoints

1. 取出与当前关键帧mpCurrentKeyFrame共视关系比较好的关键帧 ---> vpNeighKFs, (单目20, 双目10)
2. 对vpNeighKFs中每一个关键帧pKF2, 判断视差角是否足够
 - 对于双目, 相机光心距离不小于双目基线
 - 对于单目, 相机光心距离 / pKF2中MapPoints中值深度 ≥ 0.01
3. 计算mpCurrentKeyFrame与pKF2之间Fundamental矩阵 $F_{12} = K_1^{-T} [t]_{\times} R_{12} K_2^{-1}$
4. 通过SearchForTriangulation函数查找特征匹配 (借助F矩阵剔除Outlier) ---> vMatchedIndices
5. 对匹配特征点DLT法三角化恢复3D点 (对于双目, 关键帧视差角大时用三角化法恢复3D点, 反之则直接反投影) 检查3D点有效性: a. 恢复3D点 b. 检测3D点是否在相机前方 c. 重投影误差。d. 3D点到俩相机距离之比和特征点金字塔层数之比接近
6. 将3D点构造为MapPoint并mlpRecentAddedMapPoints进行一致性观测检查

局部优化

SearchInNeighbors

1. 取出与当前关键帧一级共视的关键帧与二级共视（共视关键帧的共视关键帧）的关键帧
2. 将当前帧的MapPoints分别与一级二级相邻帧(的MapPoints)进行融合。matcher.Fuse函数
 - 投影当前帧的MapPoints到相邻关键帧pKFi中，并判断是否有重复的MapPoints
 - 如果MapPoint能匹配关键帧的特征点，并且该点有对应的MapPoint，那么将两个MapPoint合并
 - 如果MapPoint能匹配关键帧的特征点，并且该点没有对应的MapPoint，那么为该点添加MapPoint
3. 将一级二级相邻帧的MapPoints分别与当前帧（的MapPoints）进行融合
4. 更新与其它帧的连接关系

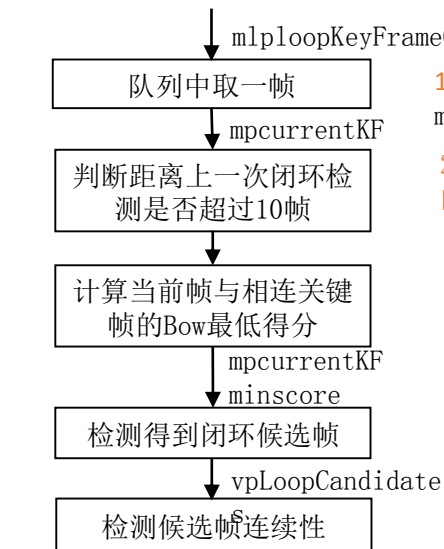
局部优化

KeyFrameCulling: 从当前共视关键帧中剔除冗余关键帧

1. 取出与当前关键帧共视的所有关键帧vpLocalKeyFrames
2. 对于vpLocalKeyFrames中每个关键帧判断是否是冗余关键帧
 - 取出该关键帧对应的MapPoints---> vpMapPoints
 - 如果vpMapPoints中有90%的MapPoints可被至少3帧尺度近似的關鍵帧观测到，则为冗余关键帧

- 预备知识
- 视觉VO与重定位
- 局部优化
- **全局闭环**

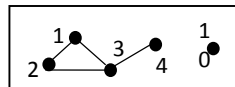
全局闭环优化



1、三个阈值都是计算获得，鲁邦性好
minscore mincommons minscoreToRetain

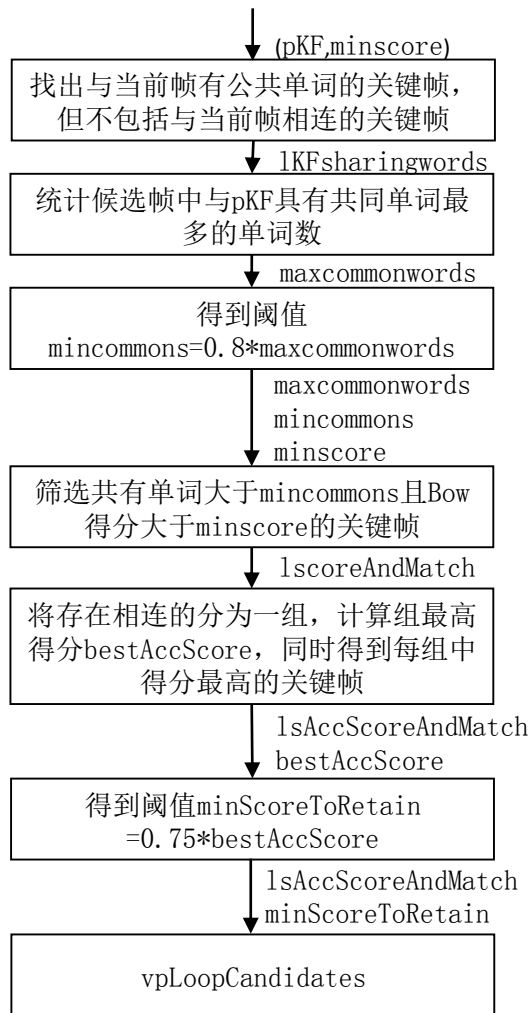
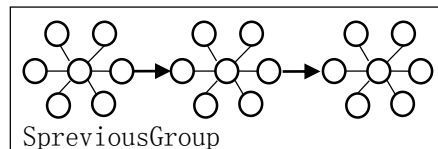
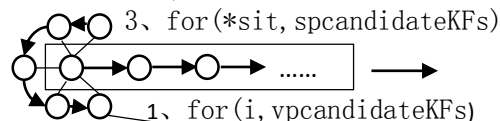
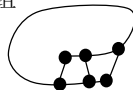
2、通过分组可以将单独得分很高的无匹配关键帧剔除

分组示意图：

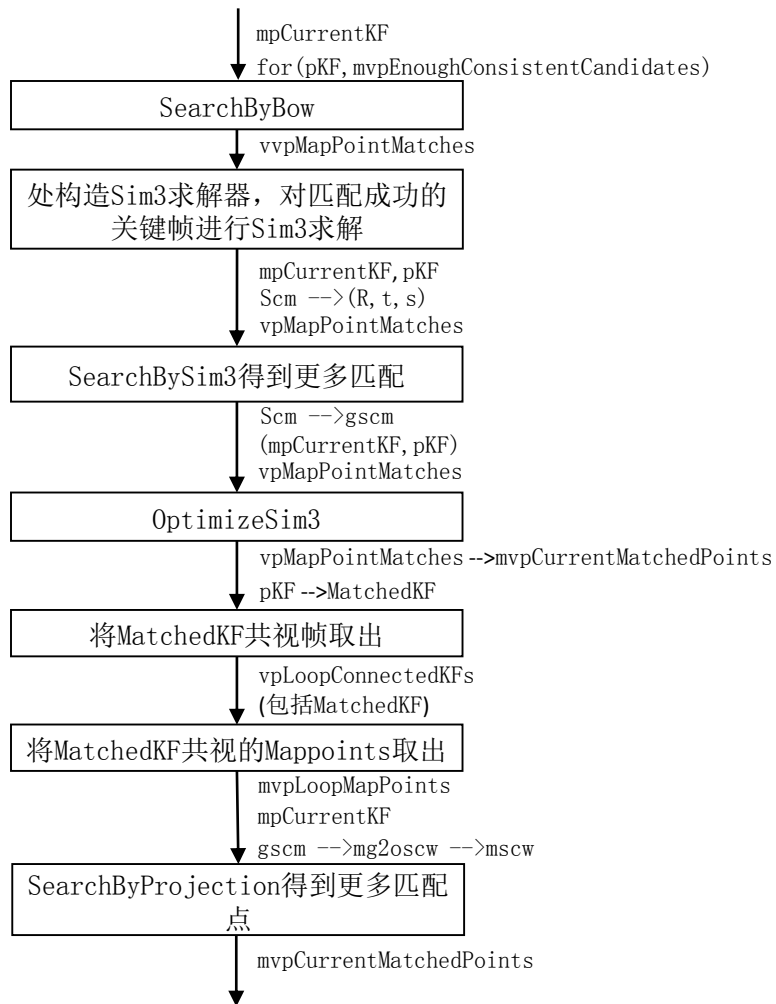


如图：1、2、3、4、10都是闭环候选帧。
节点1：与2、3相连，1与2、3分为一组
节点2：与1、3相连，2与1、3分为一组
节点3：与1、2、4相连，3与1、2、4分为一组
节点4：与3相连，4与3分为一组
节点10：10自己单独一组

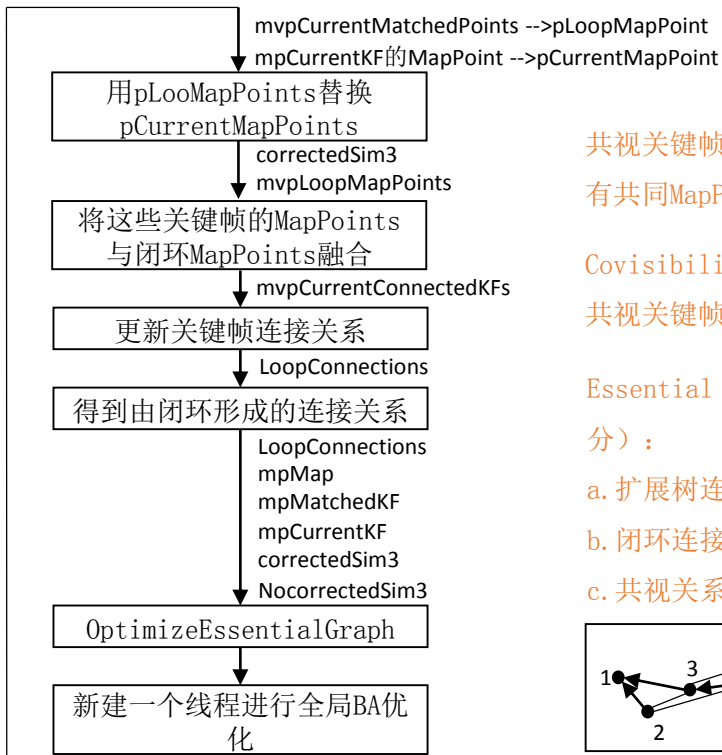
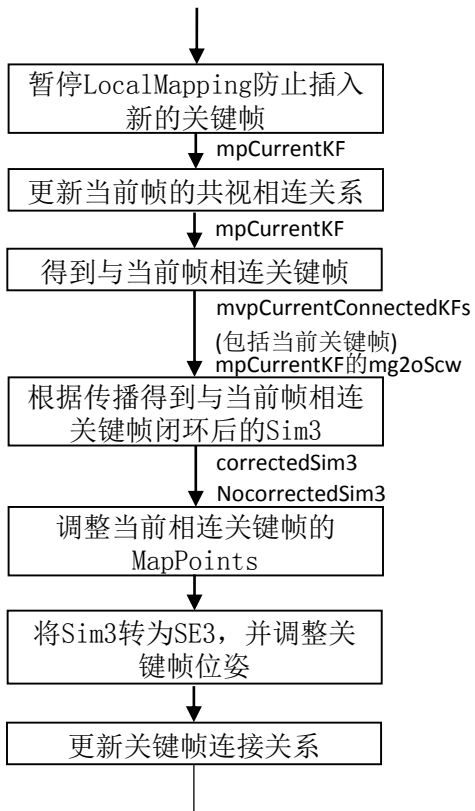
连续性检测示意图：



全局闭环优化



全局闭环优化



共视关键帧:

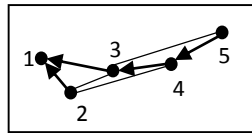
有共同MapPoints观测的关键帧

Covisibility Graph:

共视关键帧及它们的连接关系构成的图

Essential Graph (包含3部分):

- a. 扩展树连接关系
- b. 闭环连接关系
- c. 共视关系非常好的连接关系



g2o优化

GlobalBundleAdjustemnt与LocalBundleAdjustment: 同时优化3D点和位姿

3D-2D 最小化重投影误差 $e = (u,v) - \text{project}(T_{cw} * P_w)$

Vertex: `g2o::VertexSE3Expmap()`, 即当前帧的 T_{cw}

`g2o::VertexSBAPointXYZ()`, MapPoint的`mWorldPos`

Edge: `g2o::EdgeSE3ProjectXYZ()`, `BaseBinaryEdge`

Vertex: 待优化当前帧的 T_{cw}

Vertex: 待优化MapPoint的`mWorldPos`

measurement: MapPoint在当前帧中的二维位置 (u,v)

InfoMatrix: `invSigma2`(与特征点所在的尺度有关)

g2o优化

PoseOptimization: 只优化位姿

3D-2D 最小化重投影误差 $e = (u,v) - \text{project}(T_{cw} * P_w)$

只优化Frame的 T_{cw} ，不优化MapPoints的坐标

Vertex: `g2o::VertexSE3Expmap()`，即当前帧的 T_{cw}

Edge: `g2o::EdgeSE3ProjectXYZOnlyPose()`，BaseUnaryEdge

Vertex: 待优化当前帧的 T_{cw}

measurement: MapPoint在当前帧中的二维位置 (u,v)

InfoMatrix: `invSigma2`(与特征点所在的尺度有关)

Edge: `g2o::EdgeStereoSE3ProjectXYZOnlyPose()`，BaseUnaryEdge

Vertex: 待优化当前帧的 T_{cw}

measurement: MapPoint在当前帧中的二维位置 (u_l, v, u_r)

InfoMatrix: `invSigma2`(与特征点所在的尺度有关)

g2o优化

OptimizeEssentialGraph在成功进行闭环检测后，全局BA优化前进行

Vertex: g2o::VertexSim3Expmap, Essential graph中关键帧的位姿

Edge: g2o::EdgeSim3(), BaseBinaryEdge

Vertex: 关键帧的Tcw, MapPoint的Pw

measurement: 经过CorrectLoop函数步骤2, Sim3传播校正后的位姿

InfoMatrix: 单位矩阵

g2o优化

OptimizeSim3在筛选闭环候选帧时用于位姿Sim3优化

Vertex: `g2o::VertexSim3Expmap()`, 两个关键帧的位姿

`g2o::VertexSBAPointXYZ()`, 两个关键帧共有的MapPoints

Edge: `g2o::EdgeSim3ProjectXYZ()`, `BaseBinaryEdge`

Vertex: 关键帧的Sim3, MapPoint的Pw

measurement: MapPoint在关键帧中的二维位置(u,v)

InfoMatrix: `invSigma2`(与特征点所在的尺度有关)

Edge: `g2o::EdgeInverseSim3ProjectXYZ()`, `BaseBinaryEdge`

Vertex: 关键帧的Sim3, MapPoint的Pw

measurement: MapPoint在关键帧中的二维位置(u,v)

InfoMatrix: `invSigma2`(与特征点所在的尺度有关)